

BTF Viewer Statistics ■

Reference guide for the **deterministic statistics and analysis** provided by BTFViewer Desktop and Web.

Use this document when you need to answer questions such as:

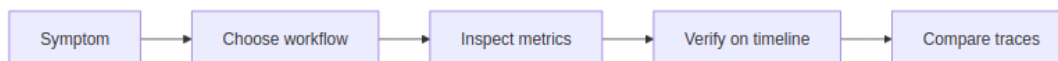
- What does this metric measure?
- How is it calculated?
- What does a high or low value mean?
- What are the limitations of the metric?
- Which related metrics should I check next?

For product usage and navigation, see [README.md](#). For step-by-step investigations, see [WORKFLOWS.md](#). For AI-assisted analysis, see [AI.md](#).

Important: BTFViewer statistics describe evidence captured in the BTF/STI trace. A maximum value is not a guarantee, correlation does not prove causation, and a missing event does not prove that the event never occurred.

Quick start

A typical investigation follows this path:



1. Start from a symptom or an **Analysis Finding**.
2. Open the related Statistics section.
3. Check **Min / Avg / p95 / p99 / Max** and the distribution, when available.
4. Jump from an unusual value or chart point to the timeline.
5. Place at least two cursors around the relevant period and enable **Limit to C1-Cn**.
6. Check related metrics before deciding on the cause.
7. Use **Trace Compare** to measure before/after changes.

What BTFViewer measures

BTFViewer statistics are based on **captured BTF/STI events and deterministic calculations derived from them**.



BTFViewer does **not** perform source-code analysis and does **not** simulate the FreeRTOS scheduler.

Some values cannot be measured exactly because the trace does not contain enough information. These values are explicitly described as **heuristic** or **limited**. For example, a true task response time requires explicit release and completion events; context-switch data alone cannot provide that relationship reliably.

Documentation map

Document	Main question	Use it for
README.md	How do I use BTFViewer?	Installation, navigation, and normal viewer usage
WORKFLOWS.md	How do I diagnose a problem?	Step-by-step investigation workflows
STATISTICS.md	What does this measurement mean?	Metric definitions, formulas, interpretation, and limitations
AI.md	How does AI-assisted investigation work?	AI setup, tools, planner, evaluation, and implementation

Statistics at a glance

The Statistics panel is shared by Desktop and Web. Most sections are collapsible. Some sections appear only when the trace contains the required STI events.

System and CPU load

Metric	What it answers
Summary	What is in the current trace or cursor scope?

Metric	What it answers
Scheduling summary	How often does each core switch tasks, and how large are the gaps?
Core utilisation	How busy is each core, and how balanced is the workload?
Trace Health (TICK)	Is the scheduler tick regular, tickless, or showing unusual gaps?
Core Time Breakdown	Where does each core spend its time: Active, Idle, Tick, or Gap?
Concurrent Core Active Distribution	How many cores are doing useful work at the same time?
Kernel Switch Overhead	How much time is spent between consecutive task slices?
Top tasks by CPU	Which tasks consume the most CPU time?

Task placement and migration

Metric	What it answers
Core Migrations	How often does a task move between cores?
Core-Pair Migration Summary	Which core-to-core migration paths are most active?
Migration & Corridor Inspector	Where are migration corridors and hot core pairs?
Core Affinity	Did a task run outside its configured affinity mask?
Task × Core	How is each task's execution distributed across cores?
Core Utilization Over Time	How does core load change over time?

Task timing and health

Metric	What it answers
Task Lifecycle	When was a task created, deleted, suspended, resumed, or dispatched?
Deadlines / CPU budget	Which slices exceed configured deadlines or CPU budgets?
Task Health	Which tasks show unusual behavior across several metrics?
Execution Time Per Slice	How long does each on-CPU slice run?
Blocking Time	How long is a task off-CPU before it runs again?
Dispatch / Scheduling Latency	How long does a recorded ready/create event wait before execution?
Inter-Arrival Time	How much time passes between consecutive task starts?
Period / Jitter	How stable is a task's observed activation period?
Response Time	What heuristic ready-to-completion delay is observed?
Unified Jitter	Which timing metric has the largest variation?

Preemption and synchronization

Metric	What it answers
Preemption Chain Analysis	Which task ran while another task was waiting?
Preemption Matrix	Which victim/preemptor pairs dominate the trace?
Priority Inheritance	When and how was task priority boosted?
Mutex / Semaphore pairing	Are synchronization events correctly paired, and where are issues?
Waiter × Owner	Which task-to-task mutex handoffs are observed?

Metric	What it answers
Mutex Blocking	Which tasks spend the most time waiting around mutex handoffs?
Queue	Are queue send/receive events paired correctly?

Instrumentation, anomalies, and comparison

Metric	What it answers
Timeline Anomalies	Which unusual events should I inspect first?
Worst Events	What are the longest timing events in the scope?
Critical Path	Where is time spent inside the longest heuristic ready-to-completion windows?
Recurring Patterns	Which anomaly types repeat for the same task?
Interval Analysis	How long do paired application-defined intervals take?
Tag Analysis	What values are recorded by tag STI channels?
Trace Compare...	What changed between two traces?
Metrics Distribution Charts	How are samples distributed over time and by value?

Panel scope and controls

The **Statistics** tab appears in the right-side panel on both Desktop and Web.

Limit statistics to a time range

Place at least two cursors, then enable **Limit to C1-Cn**. All Statistics metrics and **Analysis Findings** are recalculated for the selected range. Section titles show **(cursor range)** while this scope is active.

Clear the cursors to return to full-trace statistics.

Panel controls

Control	Action
+	Expand all sections
–	Collapse all sections; pinned sections remain open
Reset-order	Restore the default section order
Section title / chevron	Expand or collapse one section
≡ grip	Drag a section to a new position
Pin	Keep a section open when Collapse all is used

Section order, pinned sections, expanded/collapsed state, and table heights are saved across launches. **Settings → Reset to Defaults** restores the built-in layout.

Export and compare

Export CSV and **Export HTML** use the current cursor scope.

- **CSV** exports the Statistics summary tables and related calculated values.
- **HTML** exports the same summaries and adds presentation-oriented details such as **Analysis Findings**, the **Load Balance Score** gauge, and supported detail tables.
- **Trace Compare...** compares supported metrics between two open traces. Enable **Limit to each tab's cursor range** when the two traces should use different time windows.

For exact exported fields and section-specific behavior, see the detailed metric descriptions below.

Detailed metric reference

The Statistics panel is available on both Desktop and Web. Metrics are grouped into collapsible sections. Click a table column header to sort ascending or descending.

Export CSV and **Export HTML** at the panel footer honour the current cursor scope and include every section's summary table. **Export HTML** starts with an **Analysis Findings** card (same content as toolbar **Analysis** — load balance, WCET, blocking, thrashing, deadlines, tick health, and sync) and embeds the **Load Balance Score** gauge as an SVG image under Core Utilisation; use the toolbar dialog for interactive triage and **Save as Text...** HTML also adds detail sub-tables under Priority Inheritance, Mutex / Semaphore, and Interval Analysis (longest instances / hold episodes first, capped at 150-200 rows per sub-table).

Recommended workflow

1. Open a trace (e.g. `tracedata/example-4cores.btf.gz` for a 4-core SMP workload, or `tracedata/example-2cores.btf.gz` for a smaller 2-core demo).
2. Optionally click toolbar **Analysis** for a severity-tagged triage of the current scope.
3. **Core utilisation** and **Trace Health** start expanded; other sections start collapsed. Expand what you need (or use **+** / **–** at the top). That expand/collapse map is saved and restored on the next launch. Pin frequently used sections so they stay open.
4. Optionally drag **≡** grips to reorder sections for your workflow; use the reset-order icon when you want the built-in sequence back.
5. Optionally place **2+ cursors** and enable **Limit to C1-Cn** to restrict every metric (and Analysis Findings) to a time window.
6. Click a **table row** to open a distribution chart (where supported), click **Min / Max / p95 / p99** to jump and add an annotation at that slice or gap, click a **Timeline Anomalies / Worst Events** row to zoom and place C1-C2, click a **Mutex / Semaphore** issue row to zoom, jump, and annotate at that STI event, or in **Deadlines / CPU budget** click a slice row to annotate / a CPU-budget row to highlight the task (or use the **Settings → Display** link to edit thresholds).
7. Use toolbar **Compare** when two traces are open to diff summary and migration stats.

Example plots below use `tracedata/example-8cores.btf.gz`. Worked diagnosis steps for that sample: [WORKFLOWS.md §3](#).

Analysis Findings

Toolbar **Analysis** is the same heuristic card on Desktop and Web (not a Statistics panel section). Product buttons and overlays: [README](#) → [Analysis Findings](#).

Load-balance findings use the same Score / σ / Gini as **Core utilisation**. They are emitted only when there are ≥ 2 **cores** and total utilisation is > 0 . Balanced and moderate cases still show the metric line (Load Balance Score ...% (σ =...%, G =...)); “reasonably balanced” is reserved for Score ≥ 85 % and $\sigma \leq 30$ %. Dialog text uses theme colours (--fg, --analysis-ok, --analysis-warn, --analysis-err on Web; matching Desktop QLabel ink) so info findings stay readable in **dark** and **light**.

Summary, scheduling, and core utilisation

These sections open the Statistics panel in the **default** order (Summary and Scheduling summary are always first; **Core utilisation** is the first pinnable section). Write-ups below follow that catalogue: system load → migrations / affinity / lifecycle / deadlines → slice timing → preemption / sync / tags. Drag-reorder may place panel sections elsewhere.

Summary — scope-wide counts: trace span, tasks, segments, STI events. Span is $t_{\max} - t_{\min}$ in the active scope (full trace or cursor range).

Scheduling summary — per core, **context switches** count slice boundaries and **core gap** is idle time between consecutive slices on that core:

$$g_{\text{core}} = t_{\text{start},k+1} - t_{\text{end},k}$$

Large **max core gap** on a core that should be busy suggests starvation, tickless idle, or a single long-running task blocking others.

Core utilisation — per core, share of non-IDLE, non-TICK active time in scope:

$$U_{\text{core}} = \frac{T_{\text{active,core}}}{T_{\text{scope}}} \times 100$$

When two or more cores are present, two gauges are shown side by side at the top of the section — **Load Balance Score** and **Std Deviation (σ)**:

$$\text{Score} = 100 \% \times (1 - G)$$

where G is the Gini coefficient of $\{U_{\text{core}}\}$.

σ is the population standard deviation of $\{U_{\text{core}}\}$. The Score needle uses a 0–100 % scale (100 = perfect balance, 0 = single-core overload); the σ needle uses a 0–60 % scale with the warn threshold at mid-scale. Zones match toolbar **Analysis**:

Zone	Condition	UI
Red	Score < 70 %	Unbalanced chip + alert (red zone); Score gauge red
Amber	Score $\geq$ 70 % and $\sigma >$ 30 %	$\sigma > 30\%$ chip; σ gauge amber (red if $\sigma > 50\%$)
OK (green)	Score $\geq$ 70 % and $\sigma \leq$ 30 %	Green needles

Toolbar **Analysis** also warns when Score < 70 % *or* $\sigma > 30\%$, and only describes cores as “reasonably balanced” when Score $\geq 85\%$ and $\sigma \leq 30\%$. When ≥ 2 cores have total utilisation > 0, the dialog still lists **Core utilisation balance** with **Load Balance Score ...% (σ =...%, G =...)** in the OK and moderate cases (same as the gauges; Desktop and Web). **Export HTML** includes the gauges and an Analysis Findings card (info/ok ink readable on white paper); **Export CSV** includes the score, σ , and G values.

What it tells you: Imbalanced utilisation across cores may indicate poor affinity, lock pinning, or workload placement issues — cross-check with **Core Migrations**, the Migration & Corridor Inspector, and toolbar **Analysis**.

Trace Health (TICK)  Uses STI **TICK** timestamps to estimate scheduler tick regularity and detect whether the trace was recorded with the standard periodic tick or FreeRTOS **tickless idle** (configUSE_TICKLESS_IDLE).

Formula — for consecutive TICK events at times t_n :

$$\Delta_n = t_n - t_{n-1}, \quad \mu = \text{mean}(\Delta_n), \quad \sigma = \text{stdev}(\Delta_n), \quad \text{CV} = \frac{\sigma}{\mu}$$

Missed ticks (est.) counts large gaps where $\Delta n \gg \mu$ (roughly $[\Delta n / \mu] - 1$).

What it tells you: In **TICK** mode ($CV \leq 5\%$), intervals cluster at one nominal period — the scheduler clock is steady. **TICKLESS** mode ($CV > 5\%$) widens the distribution because idle periods suppress the tick interrupt; tall spikes in the scatter plot are multi-tick sleeps, not necessarily overload. Large **Max gap** with **good** status may still be worth inspecting for long critical sections or trace dropouts.

Field	Meaning
Status	good / warning / critical based on gap threshold
Mode badge	TICK (blue) or TICKLESS (amber) — detected automatically from the coefficient of variation ($CV = \sigma/\mu$) of consecutive tick intervals. $CV > 5\%$ is classified as tickless. Hover the badge to see the exact CV value
Ticks	TICK event count in scope
Avg period / Max gap	Observed tick spacing
Missed ticks (est.)	Rough count of skipped ticks from large gaps

In **tick mode** the timer interrupt fires at a constant rate and tick intervals form a tight cluster. In **tickless mode** the scheduler suppresses the tick interrupt during idle periods to save power, so consecutive intervals span one or many nominal tick periods — the distribution widens significantly.

When tick intervals can be charted, a **Tick Distribution...** button (bar-chart icon, theme-aware amber/orange styling) appears beside the **TICK / TICKLESS** mode badge when ≥ 2 ticks are in scope (Desktop + Web). Clicking it opens the standard scatter + histogram popup showing: - **Scatter plot** — each tick interval over trace time; long idle periods appear as tall spikes. - **Histogram** — interval distribution with adaptive scaling and a CDF overlay (auto may choose p5–p95 or log duration when tickless idle stretches the range); clearly multi-modal in tickless mode (one sharp peak at $1 \times$ period, another at $2\times$, $3\times$, etc.). The CDF helps quantify what fraction of tick intervals are a single period vs two or more.

Click any scatter point to jump to that tick time, add an **annotation**, and open the **Marks** tab with the annotation selected (same behaviour as other metric distribution charts).

Tick Distribution in `example-8cores.btf.gz` (~2496 TICK events, $CV \approx 35.9\%$ → **TICKLESS**):

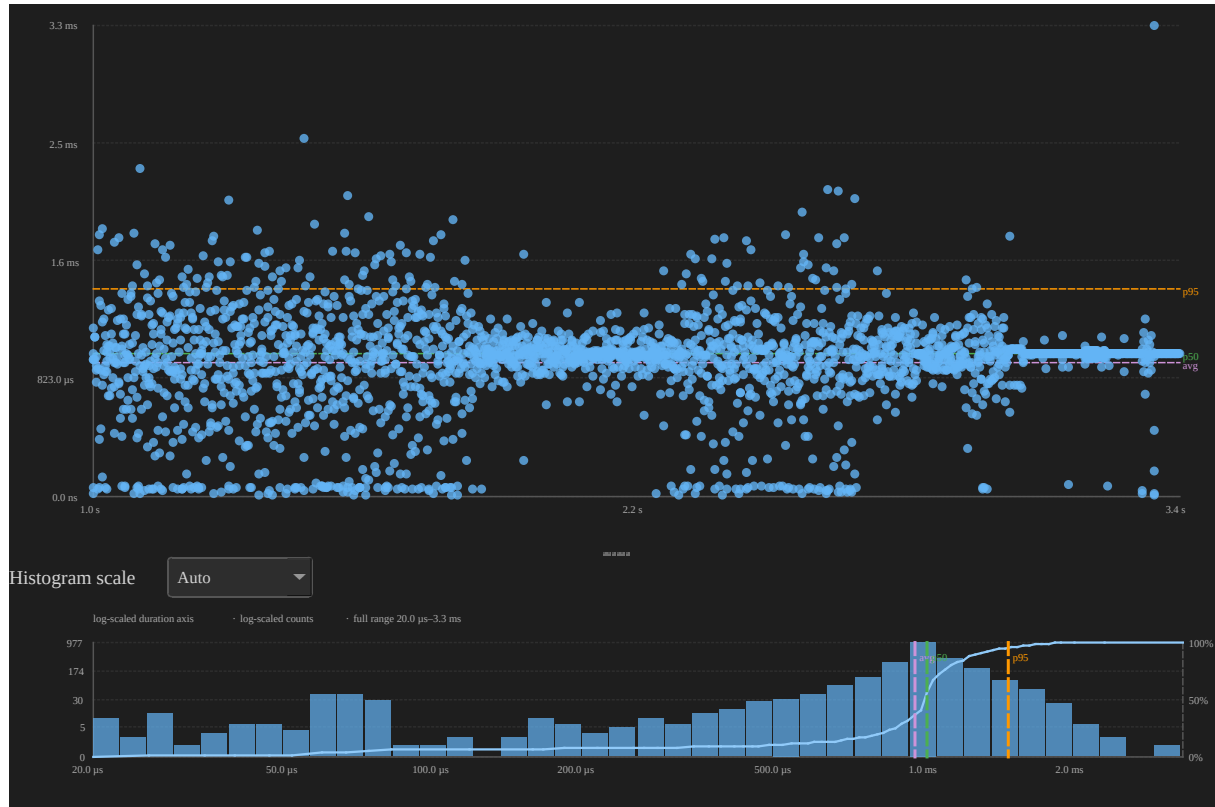


Figure 1: Tick interval distribution chart — scatter and histogram of consecutive TICK gaps in `example-8cores.btf.gz`

The histogram's multiple peaks (1×, 2×, 3× the nominal period) confirm tickless idle: most gaps are a single tick, but idle stretches skip several nominal periods before the next TICK fires.

Large gaps may indicate CPU overload, long critical sections, tickless idle, or tracing gaps — not necessarily a FreeRTOS configuration error.

Core Time Breakdown ■

Per-core time budget for the scoped window, split into four mutually exclusive buckets (percentages of the core span):

Bucket	Meaning
Active	Non-IDLE, non-TICK task time
Idle	IDLE task time
Tick	TICK handler time
Gap	Unaccounted span between consecutive core_segs (scheduler latency / ISR overhead / tracing gaps)

What it tells you: High **Gap** % on a busy core often correlates with elevated **Kernel Switch Overhead** or long ISR/critical sections. High **Idle** % with low **Active** % on some cores and overload on others points to affinity or placement imbalance — cross-check **Core utilisation** and **Core Migrations**. Click a core row (Desktop) to focus that core in **Core View**.

Concurrent Core Active Distribution

Temporal parallelism: how much of the scoped window is spent with exactly N cores concurrently running non-IDLE/non-TICK work ($N = 0 \dots \text{number of cores}$).

Formula — let $\text{isActive}(c, t)$ be 1 when core c runs a non-IDLE/non-TICK task at time t :

$$N_{\text{active}}(t) = \sum_c \text{isActive}(c, t)$$

The table aggregates dwell time at each level N .

Column	Meaning
Active Cores	Concurrency level N
Duration	Total time spent with $N_{\text{active}} = N$
% of Span	Share of the scoped window

Distribution chart — click any row: scatter of interval start vs dwell duration while

that many cores were active; histogram of those dwells. Complements **Load Balance Score** (which is utilisation balance, not simultaneous activity).

Headless snapshot (example-8cores.btf.gz, $N = 4$ active cores):

```
python builds/btf_viewer.py snapshot ../tracedata/example-8cores.btf.gz \
  -o stats-concurrency-4.svg --view plot --metric concurrency
  ↪ --active-cores 4
```

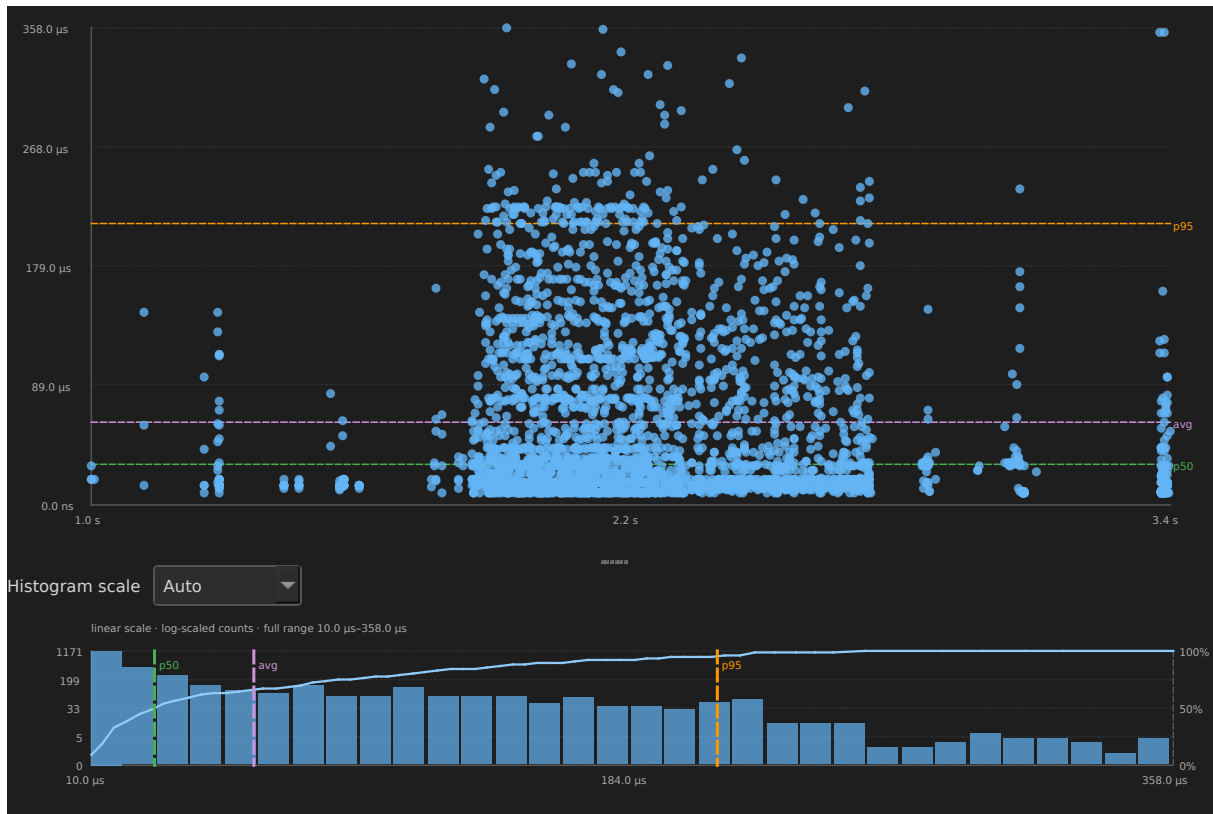


Figure 2: Concurrent core active interval-duration distribution for $N=4$ in example-8cores.btf.gz

Kernel Switch Overhead ■

Per-core cost of consecutive context switches from core_segs gaps — time from the previous slice's preempt/end to the next slice's resume/start on the same core:

$$O_{\text{switch}} = t_{\text{resume},B} - t_{\text{preempt},A}$$

Column	Meaning
Core	Core name (Core_N)
Switches	Number of consecutive-slice gaps in scope
Min / Avg / Max	Switch-gap duration statistics
Total Overhead	Sum of gaps in scope
% of Core	Total overhead as a percentage of the core / scope span

Distribution chart — click any row: scatter of switch time vs gap duration; histogram with variability overlay ($\text{avg} \pm \sigma$). Zero or near-zero gaps mean back-to-back slices with negligible scheduler overhead in the trace resolution.

Headless snapshot (example-8cores.btf.gz, Core_0):

```
python builds/btf_viewer.py snapshot ../tracedata/example-8cores.btf.gz \  
-o stats-switch-core0.svg --view plot --metric switch_overhead --core \  
  ↪ Core_0
```

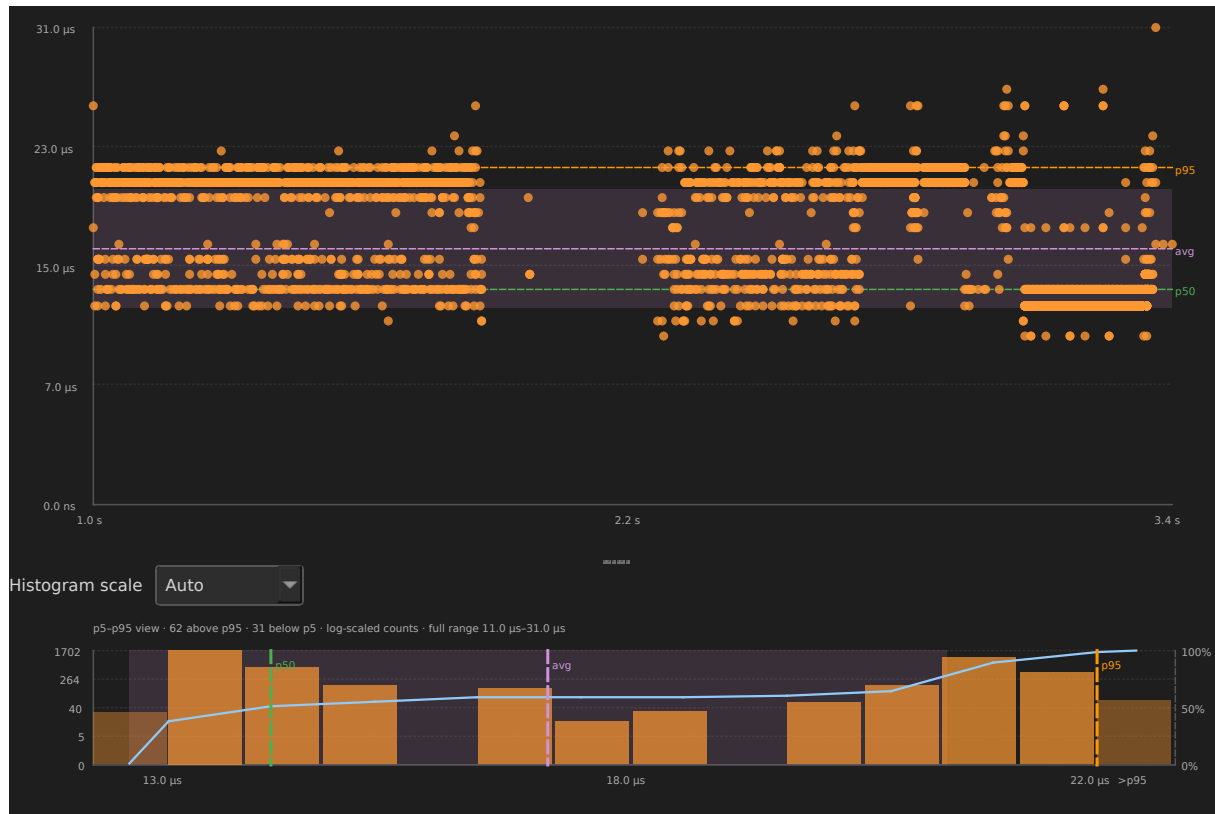


Figure 3: Kernel switch overhead distribution for Core_0 in example-8cores.btf.gz

Top tasks by CPU

Ranks worker tasks (excl. IDLE/TICK) by total on-CPU time in scope — the same $T_{\text{exec},i}$ denominator used for **CPU%** in Execution Time Per Slice. Click a row to highlight the task on the timeline.

Migration tables, highlight / inspect steps, the corridor inspector, and **Trace Compare...** are under **Core migration analysis**.

Core Affinity

Per-task affinity mask history from affinity_set STI (vTaskCoreAffinitySet) vs. observed execution cores. **Violations** lists cores used while outside the mask

then in effect (slices before the first set are unrestricted; mask changes are shown as `0x1 → 0x8`).

Always visible. Empty hint when the trace has no `affinity_set` events.

Task × Core

Per-task execution time on each core as a percent of the scoped span. Complements Core Time Breakdown (which is per-core totals) and Core Affinity (which is mask vs observed cores). Click a cell to jump to the first on-CPU slice of that task on that core.

Core Utilization Over Time

Equal time bins of the current Statistics scope, with each core's busy percent in that bin. Complements Core Time Breakdown (span totals) and Task × Core (per-task share). Click a bin to zoom that window.

Core Utilization Over Time — example-8cores								
Time	Core_0	Core_1	Core_2	Core_3	Core_4	Core_5	Core_6	Core_7
1.014 s	95.7%	92.0%	94.3%	92.7%	93.2%	93.4%	93.3%	92.2%
1.166 s	94.4%	94.6%	93.3%	93.2%	92.2%	94.2%	95.7%	93.6%
1.317 s	96.7%	95.4%	94.6%	94.4%	94.2%	96.1%	95.2%	95.1%
1.468 s	93.9%	91.2%	91.7%	93.4%	94.0%	92.9%	94.4%	94.7%
1.620 s	96.3%	93.4%	94.8%	93.6%	93.0%	95.1%	95.0%	95.4%
1.771 s	60.0%	58.5%	58.6%	70.6%	75.4%	78.5%	79.1%	80.2%
1.922 s	2.0%	2.4%	7.1%	31.4%	68.7%	70.8%	75.8%	81.8%
2.073 s	0.0%	4.2%	12.0%	26.3%	59.4%	75.7%	79.4%	83.7%
2.225 s	29.6%	29.0%	31.6%	41.5%	60.5%	73.4%	72.9%	76.7%
2.376 s	55.6%	56.5%	69.8%	76.9%	83.1%	85.4%	82.9%	88.8%
2.527 s	69.4%	63.2%	73.8%	78.9%	78.9%	83.5%	86.5%	84.2%
2.678 s	80.5%	82.4%	84.3%	84.9%	87.6%	85.3%	86.6%	86.4%
2.830 s	81.8%	80.8%	80.8%	81.7%	91.9%	81.4%	80.2%	80.8%
2.981 s	87.4%	61.0%	59.0%	60.4%	56.9%	56.2%	57.5%	58.0%
3.132 s	90.4%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.3%
3.284 s	48.9%	6.7%	6.1%	5.9%	0.9%	1.9%	3.9%	9.2%

Figure 4: Core utilization over time bins in example-8cores.btf.gz

Task Lifecycle

Per-task summary of create, delete, suspend, and resume events recorded on the task STI channel: created/deleted timestamps, suspend/resume counts, alive span (create→delete), total lifecycle event count, and **Runs** — the number of times the task was actually dispatched onto a core (context-switch-in / segment count). Runs is a scheduler-level metric and is normally much larger than Susp/Res, which only counts explicit `vTaskSuspend()`/`vTaskResume()` API calls — a task can run (and be preempted/resumed by the scheduler) many times without ever being suspended.

In example-8cores.btf.gz, test 9 creates **SR0-SR3** (pinned across cores) with overlapping suspend-while-blocked and suspend-while-running rounds — use this section to confirm Susp/Res counts match the STI pairs.

Always visible. Empty hint when the trace has no task create/delete/suspend/resume

STI events.

Deadlines / CPU budget

Configurable per-task execution deadline (**nanoseconds**, converted to the trace `#timeScale` before compare) and global CPU budget threshold (%). Shows **Slice over deadline** (top 20 by duration; click a row to jump + annotate) and **CPU budget exceeded** (click a row to highlight the task). Click column headers to sort. Click **Settings** → **Display** in the section to open Analysis thresholds (Ctrl+,). Always visible, with a configure prompt when no thresholds are set.

Task Health

A **heuristic** 0–100 score from measured statistics (execution spread, blocking tail, period CV / missed activations, migration ratio, deadline misses, CPU share). Status marks are ✓ / △ / x. This is not an AI probability. Click a band to open the matching Statistics section.

Task timing

These three task metrics are measured from consecutive on-CPU slices of the same task. The figure below shows how they relate on the timeline (UI label **Block Time** = Statistics **Blocking Time**):

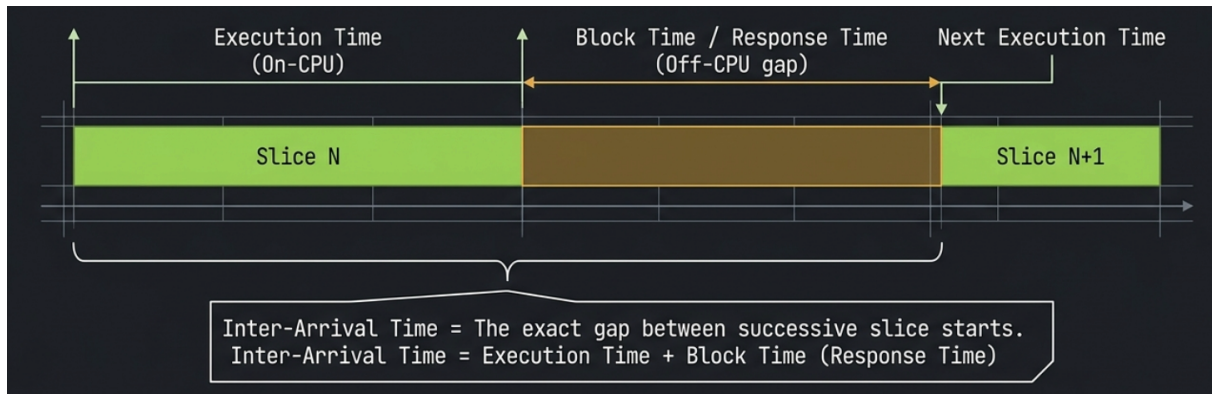


Figure 5: Execution Time, Block Time, and Inter-Arrival Time on consecutive task slices

Metric	Spans
Execution Time	Start → end of one on-CPU slice
Blocking Time	End of one slice → start of the next (off-CPU gap)
Inter-Arrival Time	Start of one slice → start of the next (period between activations)

Inter-arrival $\approx$ execution + blocking for the same pair of activations (when the gap is positive).

Timeline Anomalies ■

Scans the current Statistics scope for unusually long execution, blocking, or heuristic response tails (mean+3 σ or $\geq$ p99), tight migration / preemption / ISR / wakeup bursts, CPU utilization spikes, idle gaps, mutex-wait spikes, and configured deadline misses. Click a row to zoom, place C1-C2, highlight the task, and scroll to the matching table. **Investigate...** opens the AI tab on the selected (or top) anomaly.

Worst Events

One list of the longest execution, blocking, inter-arrival, and heuristic response episodes across tasks. Click a row to jump and set cursors on that episode.

Critical Path

Takes the longest heuristic ready→completion windows and splits each into overlapping exec / preempt / wait / migration / other. This is **not** a kernel release/completion pair. Click a component to jump to that episode.

Critical Path — example-8cores					
Task	Duration	Exec	Preempt	Wait	Mig
Runner[1]	752.699 ms	337.000 μs	704.406 ms	0.000 ns	752.362 ms
Runner[1]	361.995 ms	174.000 μs	229.497 ms	0.000 ns	395.979 ms
Runner[1]	332.613 ms	368.000 μs	267.042 ms	0.000 ns	335.737 ms
Runner[1]	109.899 ms	317.000 μs	104.979 ms	0.000 ns	109.582 ms
NC[118]	109.744 ms	109.744 ms	0.000 ns	0.000 ns	1.530 ms
Runner[1]	93.780 ms	126.000 μs	89.007 ms	4.647 ms	0.000 ns
Low[266]	67.095 ms	210.000 μs	49.026 ms	17.859 ms	0.000 ns
Runner[1]	60.715 ms	134.000 μs	3.724 ms	0.000 ns	60.581 ms

Figure 6: Critical Path table for example-8cores.btf.gz

Recurring Patterns

Groups Timeline Anomalies by task and kind and keeps kinds that repeat. Click a row to jump to the worst instance.

Recurring Patterns — example-8cores				
Task	Kind	Count	Worst	Why
Runner[1]	inter-arrival	3	754.877 ms	3× inter-arrival for Runner[1]
Runner[1]	blocking	3	752.362 ms	3× blocking for Runner[1]
Runner[1]	response	2	752.699 ms	2× response for Runner[1]

Figure 7: Recurring Patterns table for example-8cores.btf.gz

Execution Time Per Slice

Measures how long each **on-CPU slice** lasts for a task — from switch-in until the task blocks, yields, or is preempted.

Formula — for task i , each on-CPU slice k in scope:

$$d_k = t_{\text{end},k} - t_{\text{start},k}$$

Table statistics are computed over all slice durations d_k in scope. **Jitter** is the observed range, $\text{Max} - \text{Min}$; σ is the population standard deviation (the observed slices are treated as the complete population in scope). **CPU%** is the task's share of total active CPU time in scope:

$$\text{CPU}_i = \frac{T_{\text{exec},i}}{\sum_j T_{\text{exec},j}} \times 100$$

What it tells you: Short, uniform slices suggest periodic or tick-driven scheduling. A long **Max** or heavy **p95** tail marks worst-case execution time (WCET) slices — often critical sections, lock holds, or interrupt-disabled regions. Compare **Min** (BCET) and **Max** (WCET) to judge jitter; a wide spread on a real-time task may violate deadline assumptions even when **Avg** looks acceptable.

Column	Meaning
Task	Display name (Name[id])

Column	Meaning
Runs	Number of slices in scope
CPU%	Share of total trace (or cursor-range) active time
Min / Avg / Max / p95	Slice duration statistics
Jitter	Observed duration range (Max – Min)
σ	Population standard deviation of slice durations
Min / Max links	Jump and annotate BCET / WCET slice

Distribution chart — click any row:

- **Scatter:** x = slice start time, y = slice duration.
- **Histogram:** distribution of slice durations (auto log scale when the tail is wide; the blue CDF curve shows the cumulative fraction of samples).
- **Variability overlay:** the translucent purple band is average $\pm$ one population σ . Jitter (Max – Min) is the full extent of the plotted data, so it needs no marker lines.

In `example-8cores.btf.gz`, task **CS[11]** has 730 slices with a long tail of longer runs (context-switch stress tasks):

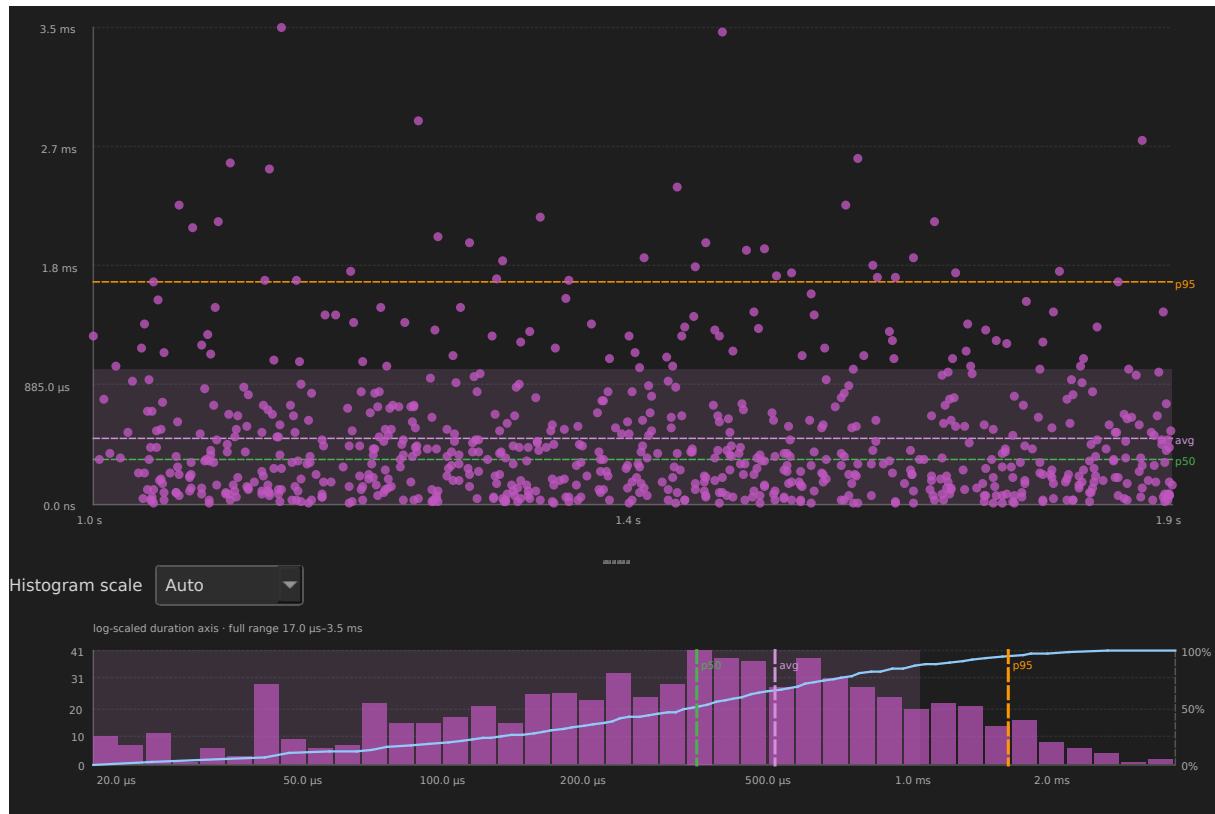


Figure 8: Execution time distribution for CS[11] in example-8cores.btf.gz

The scatter shows periodic bursts of short slices; the histogram uses a **log-scaled duration axis** so short and long slices are both visible. The **CDF** rises steeply on the left (most slices are short) then levels toward 100% as longer runs are included; **p5**, **p50**, and **p95** vertical markers align with the 5%, 50%, and 95% ticks on the right axis.

Blocking Time ■

Measures the **off-CPU gap** between the end of one slice and the start of the next for the same task — time spent waiting to run again (preempted, blocked on a resource, or delayed by the scheduler).

Not end-to-end response time: BTFViewer's **Blocking Time** is only the off-CPU gap until the task next resumes. True response time is normally measured

from an explicit release event to completion and cannot be derived reliably from context-switch slices alone. It requires matching release/completion instrumentation (for example, paired interval events).

Formula — for consecutive activations k and $k+1$ of task i :

$$g_k = t_{\text{start},k+1} - t_{\text{end},k}$$

Only positive gaps are counted. Min / Avg / Max / p95 are taken over all gaps g_k in scope.

What it tells you: Blocking time is pure **wait time** — the task is runnable or blocked but not on-CPU. High **Avg** or **Max** gaps often point to lock contention, priority inversion, or a higher-priority task monopolizing the core. Spikes clustered at specific times in the scatter plot usually correlate with a particular preemptor or synchronization object; use **Preemption Chain Analysis** or **Mutex / Semaphore** pairing to find the cause.

Column	Meaning
Task	Display name
Gaps	Number of positive off-CPU gaps
Min / Avg / Max / p95	Gap duration statistics
Jitter	Observed gap range (Max – Min)
σ	Population standard deviation of off-CPU gaps
Min / Max links	Jump and annotate resume slice at shortest / longest gap

Distribution chart — click any row:

- **Scatter:** x = resume time, y = off-CPU gap.
- **Histogram:** distribution of blocking gaps.

CS[11] in `example-8cores.btf.gz` (729 gaps):



Figure 9: Blocking time distribution for CS[11] in example-8cores.btf.gz

High blocking gaps clustered at certain times often correlate with lock contention or a higher-priority task dominating the core.

Dispatch / Scheduling Latency ■

Ready→run delay for a task: tready from STI resume Name[id] (vTaskResume) or task **create** time, and tresume from the next switch-in (segment start):

$$L_{\text{dispatch},k} = t_{\text{resume},k} - t_{\text{ready},k}$$

Sync-object wakes (give/send) are **not** attributed yet — BTF notes carry the object pointer, not the woken task id.

Column	Meaning
Task	Display name (Name[id])
Activations	Number of dispatch samples in scope
Min / Avg / Max / p95 / p99	Dispatch latency duration statistics
Jitter / σ	Observed range (Max – Min) and population standard deviation
Min / Max links	Jump and annotate the fastest / slowest dispatch

Distribution chart — click any row: scatter of dispatch time vs latency; histogram with variability overlay. Click a point to jump to the switch-in segment.

Headless snapshot (example-8cores.btf.gz, lifecycle test task **SR0[271]** — create / suspend / resume samples):

```
python builds/btf_viewer.py snapshot ../tracedata/example-8cores.btf.gz \  
-o stats-dispatch-sr0.svg --view plot --metric dispatch --task \  
  ↪ "SR0[271]"
```



Figure 10: Dispatch latency distribution for SR0[271] in example-8cores.btf.gz

Inter-Arrival Time ■

Measures the gap between **successive activation start times** of the same task (time between slice starts, not off-CPU gap).

Formula — for consecutive activations k and $k+1$:

$$\Delta t_k = t_{\text{start},k+1} - t_{\text{start},k}$$

Min / Avg / Max / p95 are taken over all inter-arrival samples Δt_k in scope.

What it tells you: Inter-arrival time reflects how often the task is **scheduled to run**, including time it spent on-CPU. For periodic tasks it should cluster near the expected period; drift or bimodality hints at missed deadlines, timer jitter, or workload-dependent release patterns. Because $\Delta t_k = dk + g_k$ (slice duration plus blocking

gap), inter-arrival is always $\geq$ blocking time for the same activations — compare both tables to see whether jitter comes from short runs or long waits.

Column	Meaning
Task	Display name
Runs	Number of inter-arrival samples
Min / Avg / Max / p95	Gap between activation starts
Jitter	Observed inter-arrival range (Max – Min)
σ	Population standard deviation of inter-arrival times
Min / Max links	Jump and annotate activation at shortest / longest inter-arrival

Distribution chart — click any row:

- **Scatter:** x = activation time, y = gap since previous activation.
- **Histogram:** distribution of inter-arrival gaps.

CS[11] in `example-8cores.btf.gz`:

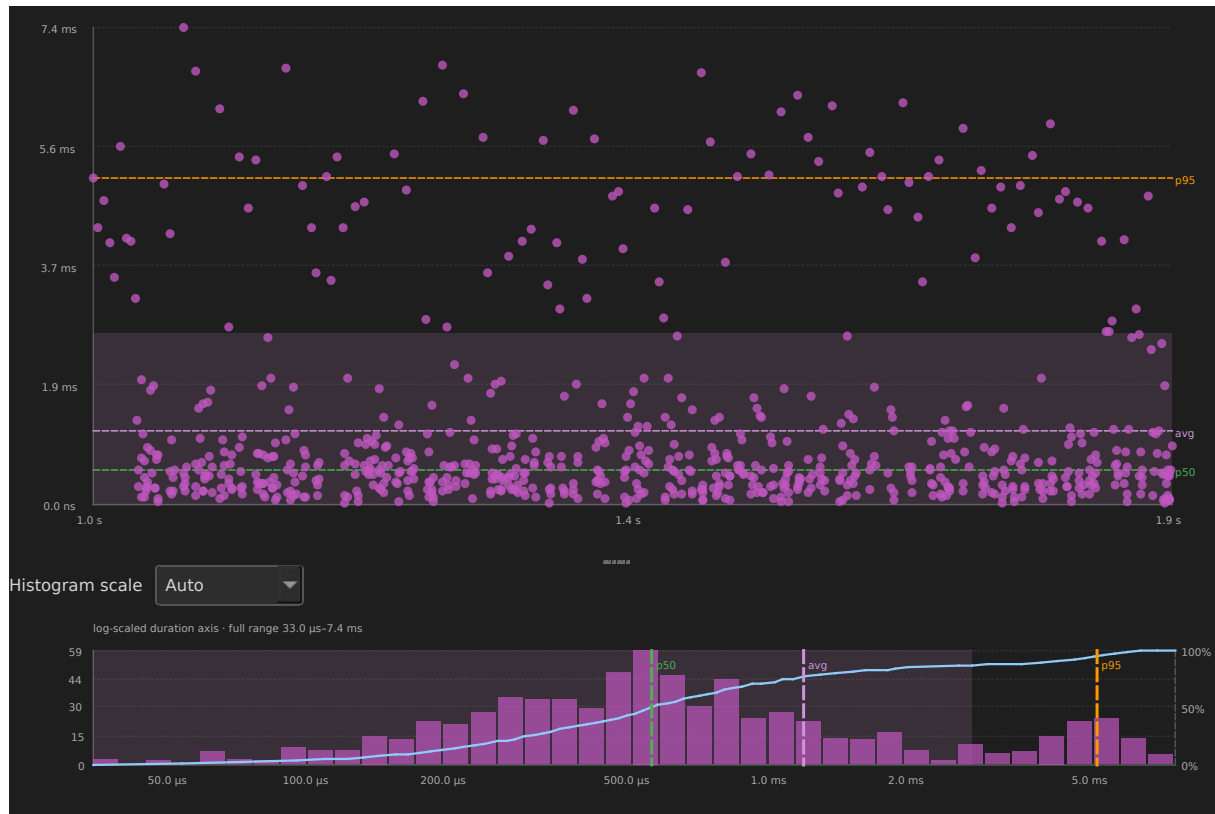


Figure 11: Inter-arrival time distribution for CS[11] in example-8cores.btf.gz

Compare with Blocking Time: inter-arrival includes time the task was **running**, so values are typically larger than off-CPU gaps alone. The histogram auto-selects **log duration** for CS[11] because activation gaps span microseconds to milliseconds.

Period / Jitter ■

Uses the same inter-arrival gaps as the Inter-Arrival table. **Expected** is the median gap. **Missed** counts gaps $> 1.5 \times$ expected; **extra** counts gaps $< 0.5 \times$ expected; **burst** counts gaps $< 0.25 \times$ expected. **RMS** is jitter versus that expected period. **Spark** is the chronological gap series. Click a time column to jump to that gap; click the task name to open the existing Inter-arrival distribution plot (this page does not add a second histogram).

Response Time ■

Heuristic ready→completion: previous slice end → this slice end (the first slice uses its own exec duration). BTF does not record an explicit release/completion pair, so this is **not** kernel response time. Percentiles include p50 / p90 / p95 / p99 / p99.9. Click the task to open the existing Response plot; click **Min** / **Max** / **p50** / **p90** / **p95** / **p99** / **p99.9** to jump to that event.

CS[11] in `example-8cores.btf.gz`:

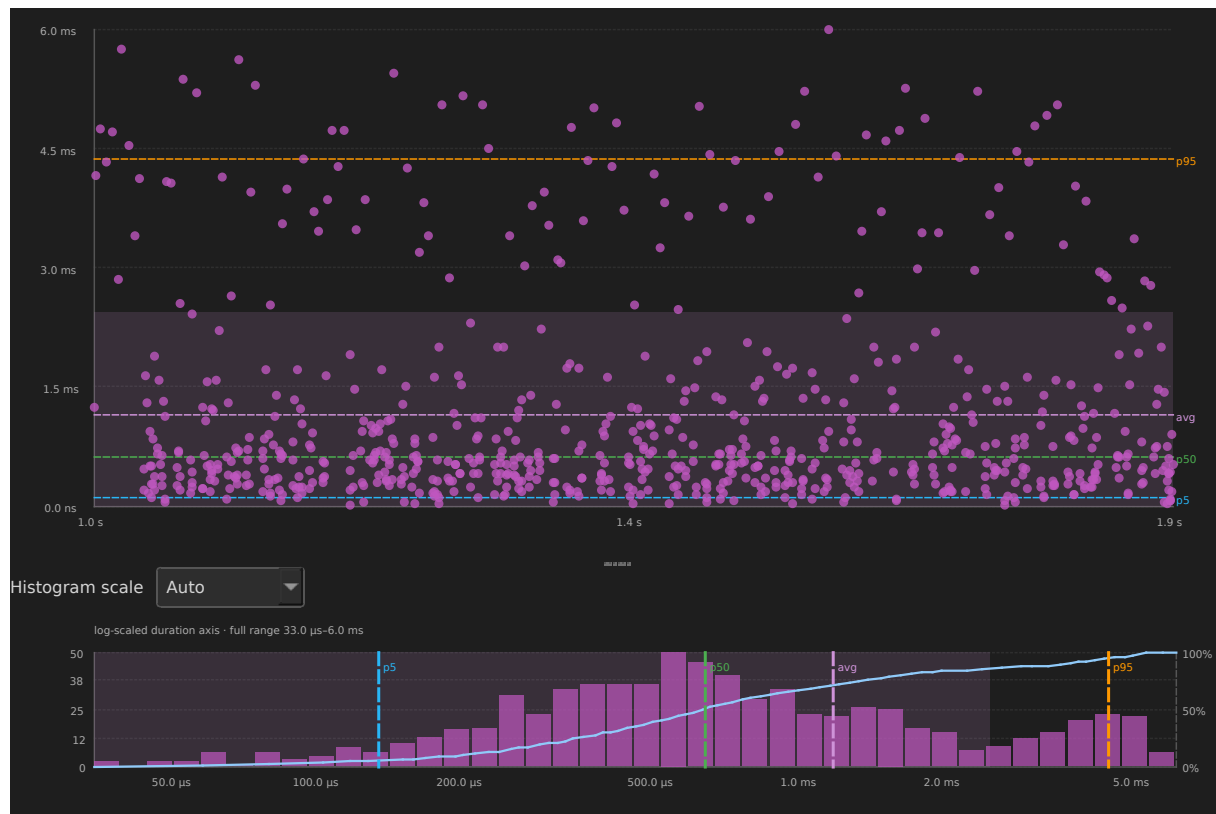


Figure 12: Response time distribution for CS[11] in example-8cores.btf.gz

Response Time — example-8cores						
Task	N	Avg	p50	p99	Max	CV
Runner[1]	346	6.996 ms	140.000 μ s	109.899 ms	752.699 ms	701.2%
NC[118]	15	9.224 ms	1.578 ms	109.744 ms	109.744 ms	292.6%
High[268]	7	32.157 ms	34.945 ms	53.045 ms	53.045 ms	55.5%
PS[228]	3	10.546 ms	2.348 ms	29.146 ms	29.146 ms	125.0%
QP[197]	22	4.105 ms	1.840 ms	25.780 ms	25.780 ms	140.5%
QP[194]	9	8.203 ms	5.171 ms	21.258 ms	21.258 ms	82.6%
TL[305]	100	602.000 μ s	31.000 μ s	20.000 ms	20.021 ms	537.7%
QP[195]	14	5.884 ms	4.827 ms	18.254 ms	18.254 ms	76.6%

Figure 13: Response Time table for example-8cores.btf.gz

Unified Jitter ■

Max–Min spread and CV for execution, blocking, inter-arrival, heuristic response, STI **dispatch** latency (resume/create → switch-in), and **wake** (heuristic response wait — the stand-in when BTF does not name the woken task). Click a column to open the matching plot. This page does not add a second histogram.

Unified Jitter — example-8cores								
Task	Exec	Exec CV	Block	Block CV	Inter	Inter CV	Response	Resp CV
Runner[1]	3.488 ms	200.5%	752.349 ms	720.6%	754.848 ms	702.9%	752.670 ms	701.2%
NC[118]	109.660 ms	321.4%	2.134 ms	76.0%	110.725 ms	286.6%	109.358 ms	292.6%
Low[266]	271.000 μ s	12.2%	66.872 ms	1236.8%	66.814 ms	834.3%	67.014 ms	836.7%
High[268]	358.000 μ s	56.4%	38.723 ms	35.5%	39.029 ms	35.5%	52.830 ms	55.5%
Med[267]	327.000 μ s	17.6%	35.297 ms	1517.9%	35.242 ms	773.8%	35.381 ms	776.4%
PS[228]	175.000 μ s	43.4%	26.623 ms	85.5%	26.499 ms	84.0%	29.001 ms	125.0%
QP[197]	17.323 ms	161.5%	8.188 ms	120.4%	19.293 ms	111.8%	25.265 ms	140.5%
TL[305]	117.000 μ s	80.5%	19.948 ms	554.7%	19.970 ms	536.1%	19.991 ms	537.7%

Figure 14: Unified Jitter table for example-8cores.btf.gz

Distribution Explorer

Choose a metric (execution, blocking, inter-arrival, response, dispatch, wake, pre-emption) and a task. The section shows n / p50 / p99 / CV, a sparkline, and the **histogram/CDF at the bottom** of the section (same adaptive scale as other metric plots). **Open histogram** still opens the full scatter + histogram window. Wake is heuristic response wait, not a kernel wakeup event.

Preemption Chain Analysis

For each **victim** task's off-CPU gap, the analyser finds which **preemptor** tasks ran on the **same core** as the victim during that gap and aggregates overlap duration.

Formula — for victim v and preemptor p on the same core during gap g :

$$\text{overlap}(v, p, g) = \sum_{p \in g} [\min(t_{\text{end}}, g_{\text{end}}) - \max(t_{\text{start}}, g_{\text{start}})]$$

Count is the number of such overlap events; **Total / Avg / Max** summarise overlap durations for each victim←preemptor pair.

What it tells you: Answers *who ran while this task waited*. A single pair with high **Count** and **Total** means one preemptor dominates the victim's blocking; many pairs with moderate counts suggest fair sharing or frequent context switches. Large **Max** overlap points to long stretches where the victim was ready but could not run — a common sign of priority misconfiguration or a CPU hog.

Column	Meaning
Victim	Task that was off-CPU
Preemptor	Task that ran during the victim's gap
Count	Number of preemption overlap events
Total / Avg / Max	Overlap duration (how long the preemptor held the CPU during victim gaps)

Distribution chart — click any row (victim ← preemptor pair):

- **Scatter:** x = when the preemption overlap started, y = overlap duration.
- **Histogram:** how long that preemptor typically held the CPU during gaps.
- **Click a point** to jump to the **preemptor's segment** and add an annotation with duration/time notes.

CS[24] ← CS[25] in `example-8cores.btf.gz` (55 overlap events, 14.936 ms total overlap) — two context-switch stress tasks repeatedly preempting each other:

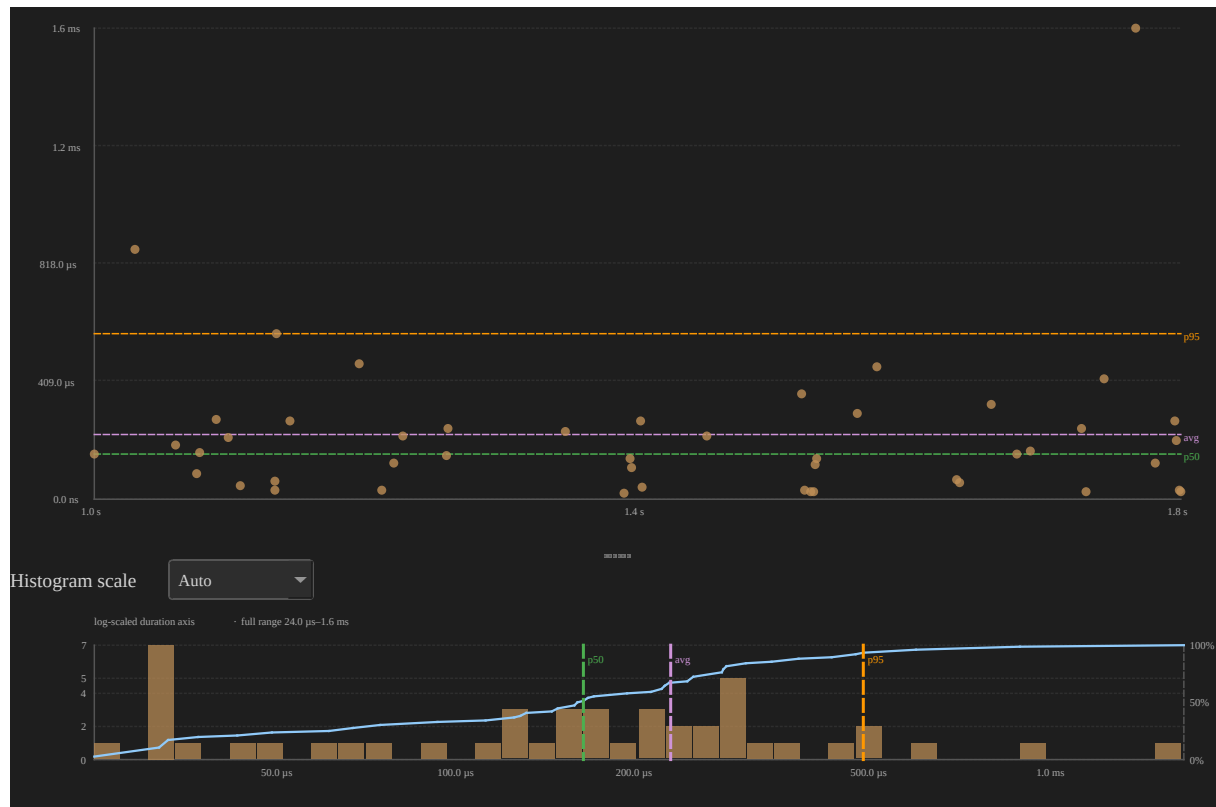


Figure 15: Preemption chain distribution CS[24] preempted by CS[25] in `example-8cores.btf.gz`

High **Count** with moderate **Avg** overlap suggests frequent short preemptions; a few points with large **y** values are long stretches where CS[25] ran while CS[24] waited. Use this table to answer *who preempted whom* and whether a victim's blocking is dominated by one preemptor or many.

Preemption Matrix

Same-core victim × preemptor overlap plus a ranking of who preempts each victim most, including an A → B → resumed story. Complements Preemption Chain Analysis (pair totals with a plot). Click a ranking row or matrix cell to jump to the longest overlap.

Preemption Matrix — example-8cores				
Victim	Count	Total	Max	Top preemptors
Runner[1]	4743	1.857 s	17.374 ms	Med[267] (1161), CS[13] (149), CS[26] (125), CS[17] (114)
CS[13]	1438	478.846 ms	2.656 ms	CS[23] (101), CS[25] (98), CS[27] (94), CS[11] (93)
CS[21]	1436	452.465 ms	3.187 ms	CS[26] (100), CS[28] (95), CS[27] (92), CS[14] (91)
CS[12]	1436	451.359 ms	3.126 ms	CS[21] (98), CS[22] (95), CS[16] (93), CS[14] (89)
CS[14]	1476	451.027 ms	2.935 ms	CS[21] (109), CS[17] (102), CS[16] (96), CS[24] (95)
CS[15]	1533	448.426 ms	2.179 ms	CS[26] (105), CS[16] (101), CS[11] (100), CS[21] (98)
CS[23]	1426	446.020 ms	2.615 ms	CS[17] (104), CS[19] (97), CS[25] (93), CS[14] (92)
CS[17]	1495	442.174 ms	2.466 ms	CS[19] (101), CS[22] (96), CS[23] (96), CS[26] (95)

Figure 16: Preemption Matrix ranking for example-8cores.btf.gz

Priority Inheritance

Shown when the trace has **create pri:N** on task-create T rows **and** at least one priority STI on the task channel:

STI note prefix	Hook	Meaning
priority_inherit Name[id] pri:N	traceTASK_ PRIORITY_INHERIT	Mutex holder inherited priority <i>N</i>
priority_disinherit Name[id] pri:N	traceTASK_ PRIORITY_ DISINHERIT	Mutex holder returned to base priority <i>N</i>
set_priority Name[id] pri:N	traceTASK_ PRIORITY_SET	Explicit vTaskPrioritySet()

Formula — a **boost episode** is a contiguous interval where the task's effective priority is above its **Base** (from `create pri:N`):

$$T_{\text{boosted}} = \sum_{\text{episodes}} (t_{\text{end}} - t_{\text{start}})$$

where t_{start} is the inherit / set-up STI and t_{end} is the disinherit / set-back STI for each episode.

Boosts counts episodes; **Peak** is the highest priority observed while boosted.

What it tells you: Priority inheritance prevents priority inversion when a low-priority mutex holder blocks a high-priority waiter. Long **Boosted** time or many **Boosts** on a task that is not the mutex owner under test may indicate lock contention or chained inheritance. **Mutex inherit** confirms the kernel raised priority via `priority_inherit`; **L/M/H pattern** flags the classic three-level inversion geometry (see below); **Boost only** means manual `vTaskPrioritySet()` without mutex hooks.

Column	Meaning
Task	Mutex holder or subject whose priority was raised
Base	Priority at task create (<code>create pri:N</code>)
Peak	Highest <code>set_priority</code> level observed while boosted
Boosts	Number of boost episodes (base → above base → back to base)
Boosted	Total time above base priority in scope
Pattern	Summary label — see L/M/H pattern and table below

Pattern (summary)	When it appears
Mutex inherit	At least one boost episode used <code>priority_inherit</code> / <code>priority_disinherit</code> , and no extra inversion episodes beyond inherits
Mutex inherit + L/M/H	Mutex inheritance and additional boost episodes where medium-priority tasks sit between base and peak

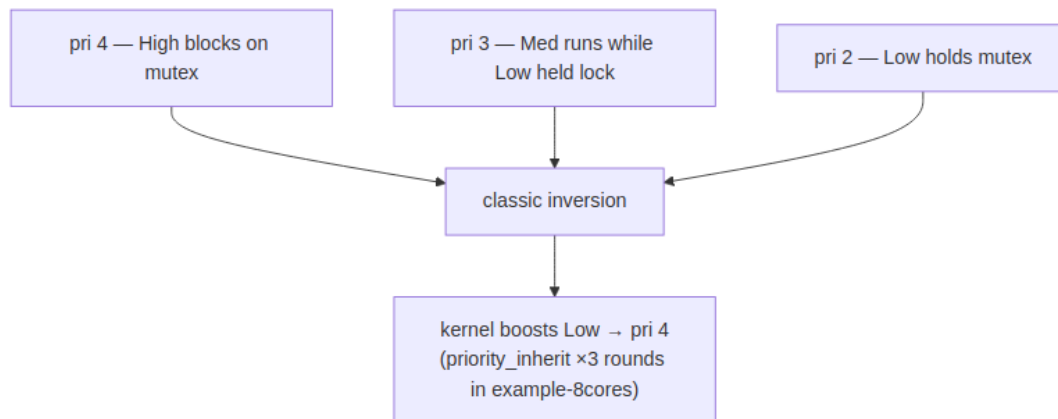
Pattern (summary)	When it appears
L/M/H pattern	Boost above base (usually set_priority) with medium-priority tasks between Base and Peak , but no priority_inherit on that task
Boost only	Raised above base with no medium tasks in range and no mutex inherit

Per-episode detail (distribution chart tooltips, **Export HTML → Boost episodes**) can be more specific, e.g. Mutex inherit L/M/H (CS[11], CS[12] +126) — listing up to two medium-task names plus a count of any others.

L/M/H pattern (priority inversion geometry) ■ **L/M/H** names the textbook **priority-inversion** layout on three priority levels. In the demo firmware (Demo/examples/freertos_test/main.c test 8) the tasks are named **Low**, **Med**, and **High**, pinned to **Core_0** on SMP, and the scenario is repeated **T8_ROUNDS (3) times** so the timeline shows multiple red boost stripes and the Priority Inheritance chart has several inherit points:

Role	Meaning	In example-8cores.btf.gz test 8
L (Low)	Mutex holder at the lowest priority of the three	Low[266] — create pri:2, holds the test mutex
M (Medium)	Runnable work at a priority between L and H; can preempt L while H waits	Med[267] — create pri:3, CPU work after Low takes the mutex
H (High)	Blocked on the mutex L holds; triggers inheritance	High[268] — create pri:4, blocks on the same mutex

Without mutex priority inheritance, **M** would run while **H** waited on **L** — unbounded inversion. FreeRTOS boosts **L** to **H**'s priority (e.g. first priority_inherit Low[266] pri:4 near ~3099 ms) so **L** finishes its critical section before **M** can starve **H**.



How the viewer detects L/M/H: at the end of each boost episode, it scans every task with a known create pri:N. Any task whose **base priority** satisfies **Base** < p < **Peak** (strictly between the boosted task's base and peak) is a **medium blocker**. If at least one exists, the episode is an **inversion suspect** and contributes to the **L/M/H** pattern.

- **Episode** level: inversionSuspect when the episode was mutex-inherited **or** medium blockers exist; episode **Pattern** may list medium task names.
- **Summary** level (table **Pattern** column): aggregates episodes — Mutex inherit, L/M/H pattern, Mutex inherit + L/M/H, or Boost only.

Important: medium-blocker detection uses **all** tasks in the trace with recorded create priorities, not only tasks active in that instant. On a busy SMP trace (many workers at pri:3 from earlier tests), the episode label may show several medium names (e.g. Mutex inherit L/M/H (CS[11], CS[12] +126)). For test 8, **Med[267]** is the semantically relevant **M**; cross-check with the timeline around test 8 (~3042-3359 ms absolute, --lo 3042000 --hi 3359000) and Demo/examples/freertos_test/main.c test 8 (vInvLow / vInvMed / vInvHigh).

Example — test 8 in example-8cores.btf.gz (Low[266]):

Field	Value
Base / Peak	2 → 4
Boosts / Boosted	3 mutex-inherit episodes, ~34.4 ms each (~103 ms total)
Summary Pattern	Mutex inherit

Field	Value
Episode Pattern	Mutex inherit L/M/H (medium tasks include Med[267] at pri 3)
STI windows	3099448→3133873, 3187403→3221850, 3275380→3309826 μ s

Zoom the timeline to that window (or click a **Low[266]** scatter point) to see the **three red** boost stripes on the Low row and **High** blocking on the mutex between them.

Contrast — test 7 (PS[228], manual boost): the runner calls `vTaskPrioritySet(subject, BOOST_PRIORITY)` — trace shows `set_priority STI` events, not `priority_inherit`. **PS[228]** has the same numeric geometry (base 2, peak 4, medium fillers at pri 3) so the summary **Pattern** is **L/M/H pattern** (not **Mutex inherit**). Boost duration is much shorter ($\sim 119 \mu$ s) because no mutex hold loop is involved. Use **PS[228]** vs **Low[266]** to separate kernel inheritance from application-driven priority changes.

Timeline UX — boosted periods appear as a **bottom stripe** on the task row (horizontal) or a **right-edge stripe** (vertical): **orange** = boost only (Boost only / manual `set_priority` without L/M/H geometry); **red** = mutex inherit or any L/M/H-related pattern. Task labels show `• pri N` when create priority is known.

Low[266] on the timeline in `example-8cores.btf.gz` (zoomed to test 8, `--lo 3042000 --hi 3359000`). Three **red bottom stripes** on the Low row mark the `priority_inherit` → `priority_disinherit` windows (~ 34.4 ms each). **Med[267]** runs on the same core before each inherit; during each boost window Med stays off-CPU:

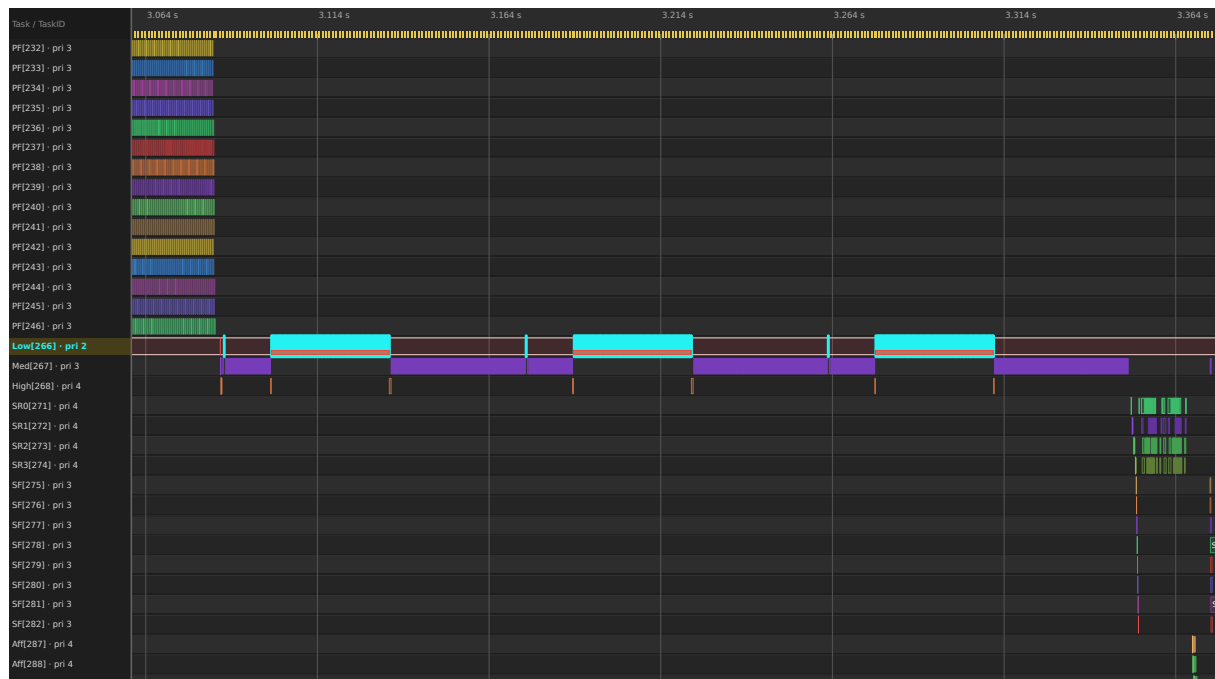


Figure 17: Timeline view: Low[266] with three red priority-inheritance stripes (example-8cores.btf.gz)

Export a timeline SVG from the viewer (**File** → **Save SVG** or toolbar) after zooming to the episode, or regenerate it headlessly via `make -C BTFViewer update-images` (desktop CLI snapshot `--view timeline --task ... --lo ... --hi ...`).

Distribution chart — click any Priority Inheritance table row:

- **Scatter:** x = boost episode end time, y = boosted duration. Orange points = boost only; red = L/M/H / mutex inherit. Test 8 contributes **three** clustered red points near ~34 ms.
- **Histogram:** distribution of boost durations.
- **Click a point** to zoom to that episode, scroll to the task row, highlight it, and add an annotation (re-click skips duplicate annotations).

Export HTML includes a **Boost episodes** detail sub-table (up to 200 rows by start time).

Low[266] distribution (test 8: three mutex inherit episodes, base pri 2 → peak 4, ~34.4 ms each):

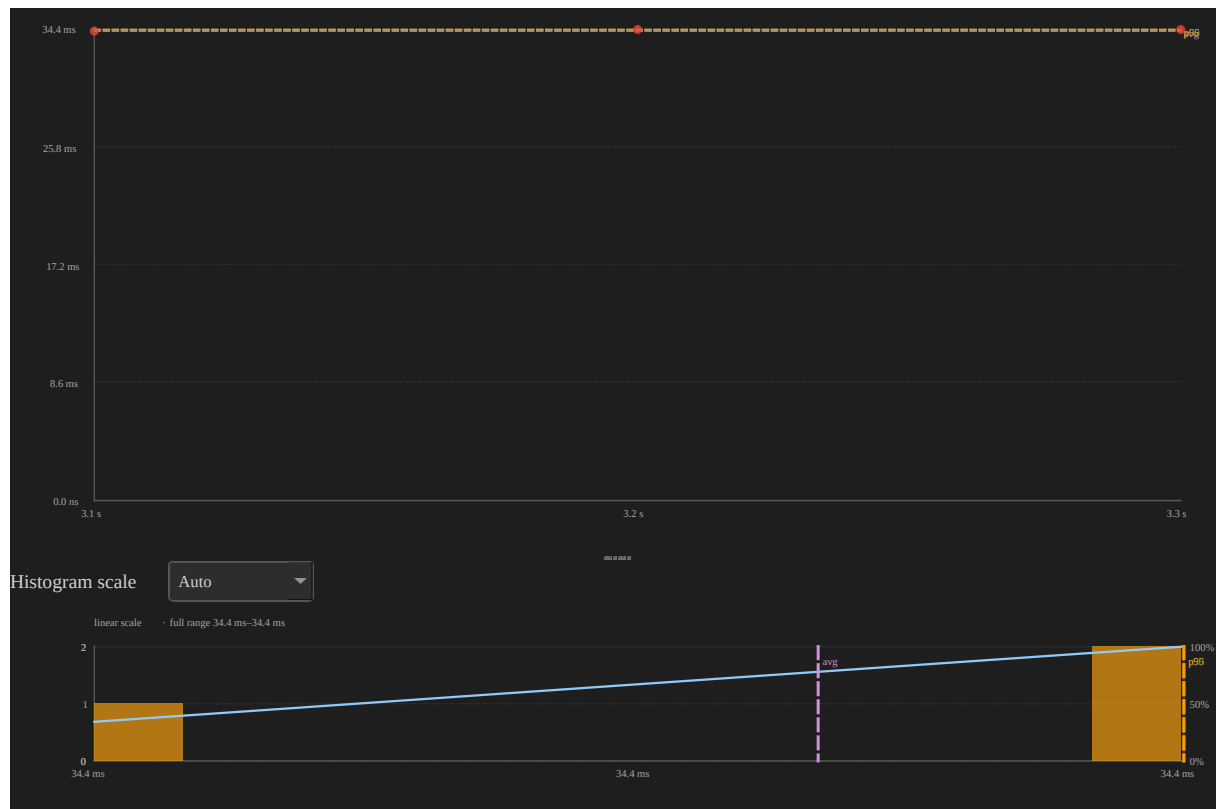


Figure 18: Priority boost distribution chart for Low[266] in example-8cores.btf.gz

PS[228] (test 7: manual vTaskPrioritySet, **L/M/H pattern**, ~119 μ s boosted) contrasts with **Low[266]** above — same base/peak numbers, different mechanism and duration.

Note: traceTASK_PRIORITY_INHERIT / traceTASK_PRIORITY_DISINHERIT are invoked by the FreeRTOS kernel inside xTaskPriorityInherit() / xTaskPriorityDisinherit() when configUSE_MUTEXES is enabled.

Mutex / Semaphore pairing ■

When queue trace hooks are enabled (configINCLUDE_QUEUE_EVENTS), mutex and semaphore STI lines include the **FreeRTOS object pointer** in the note:

```
703266,Core_0,0,STI,mutex,0,trigger,take 0x80018700
707222,Core_3,0,STI,mutex,0,trigger,give 0x80018700
```


The viewer pairs **take/give** (and **create/delete**) **per pointer** — not per STI channel alone — so two mutexes of the same type stay distinct. The kernel **give** immediately after **create** (mutex / binary sem available) is ignored when it falls within **1 ms** of the create event.

Semaphores use two pairing directions automatically:

Pattern	Example	Pairing
Hold (take → give)	Area slot sem, worker acquires then releases	take opens, matching give closes (FIFO)
Signal (give → take)	*_done / *_go coordination sems	give posts, matching take consumes (FIFO)

Mutexes always use hold pairing (LIFO — owner must give).

Formula — for each paired hold span h of duration τ_h :

$$\bar{\tau}_{\text{hold}} = \frac{1}{N_{\text{holds}}} \sum_h \tau_h$$

What it tells you: **Avg hold** shows typical lock or semaphore residency time — very long holds inflate blocking for waiters. **Issues** > 0 (orphan give, cross-task give, unmatched take, delete while held) mean the trace does not form clean take/give pairs and hold statistics may be incomplete. **Deadlock risk** at trace end flags multiple mutexes still held by different tasks — verify whether that is expected teardown or a real stall.

Column	Meaning
Object	Kind + pointer (mutex 0x80018700)
Kind	mutex or sem
Holds	Number of paired take→give spans in scope
Issues	Pairing problems in scope
Avg hold	Mean hold duration across paired spans
Status	OK , Warning , or Error

Check	Severity	Meaning
Orphan give	Error	Mutex give with no matching open take on that pointer
Cross-task give	Warning	Mutex give by a different task than the one that take
Unmatched take	Warning	take still open at trace end
Unmatched give	Warning	Semaphore give still unmatched at trace end
Delete while held	Warning	delete while resource takes are still open (common during teardown)
CORE_MIGRATION_WHILE_HELD	Warning	Mutex take on one core, give on a different core — the lock crossed a core boundary while held; indicates a cache-line bounce where the hardware cache line containing the lock data is transferred between cores. Shown in the Pairing Issues sub-table with the exact from/to cores (e.g. Lock bounced from Core_0 to Core_1)
Deadlock risk	Warning	≥ 2 mutexes still held by ≥ 2 different tasks at trace end

The running task for each event is inferred from the **core timeline** at that timestamp (same approach as interval tid pairing).

Below the summary table, a **Pairing issues** sub-table lists every problem in scope (time, object, issue kind, detail). **Click any issue row** to zoom to the running task segment on that core (when found), jump to the issue timestamp, highlight the segment, and add an annotation with a descriptive note (Desktop + Web). Re-clicking the same point skips duplicate annotations.

Export HTML adds two detail sub-tables under this section: all **Pairing issues** in scope (including CORE_MIGRATION_WHILE_HELD warnings), and **Hold episodes** (longest first, up to 150 rows) with **Take core** and **Give core** columns.

Export CSV includes a **Core Affinity Violations** sub-section listing each mutex that had at least one bounced hold, with the bounce count and a description.

example-4cores.btfgz (tests 1-3) exercises 0x80018700 (mutex) and 0x80018650 (counting sem) with clean hold pairing. The full trace shows CORE_MIGRATION_WHILE_HELD warnings on 0x80018700 (3 bounces) and 0x80018650 (1 bounce) — the Statistics **Mutex / Semaphore** summary table will show **Warning** for these mutexes and non-zero values in the **Bounces** column. Coordination sems pair in **signal** direction.

Waiter × Owner

Heuristic matrix from consecutive mutex holds on the same object: the next distinct acquirer is treated as the waiter, and the previous holder as the owner. Cell values are the summed hold time of those handoffs. This is **not** a kernel wait-queue reconstruction — BTF records successful take/give, not blocked attempts. Click a cell to zoom the longest handoff.

Mutex Blocking

Per-task totals of those heuristic mutex waits (object, last owner, count, total, max), plus a **Top blocking contributors** ranking across mutex waits, preemption overlap, and leftover idle gaps. Click a row to jump to the longest wait.

Mutex Blocking — example-8cores					
Task	Object	Owner	Count	Total	Max
High[268]	mutex:0x80021ad0	Low[266]	3	143.857 ms	47.965 ms
SM[87]	mutex:0x80021b80	SM[93]	48	17.164 ms	1.374 ms
SM[83]	mutex:0x80021b80	SM[96]	48	17.107 ms	1.003 ms
SM[97]	mutex:0x80021b80	SM[94]	48	16.603 ms	928.000 µs
SM[94]	mutex:0x80021b80	SM[88]	48	16.355 ms	1.229 ms
SM[90]	mutex:0x80021b80	SM[84]	48	16.011 ms	974.000 µs
SM[93]	mutex:0x80021b80	SM[94]	48	15.994 ms	1.005 ms
SM[89]	mutex:0x80021b80	SM[84]	48	15.947 ms	783.000 µs

Figure 19: Mutex Blocking table for example-8cores.btfgz

Queue

When queue STI events are present (configINCLUDE_QUEUE_EVENTS), the **Queue** section pairs send/recv (and create/delete) **per object pointer**, the same way Mutex / Semaphore pairing works for take/give.

Column	Meaning
Object	Kind + pointer (queue 0x.....)
Holds	Number of paired send→recv (or equivalent) spans in scope
Issues	Pairing problems in scope
Avg hold	Mean hold duration across paired spans
Status	OK, Warning, or Error

Shown only when the trace contains queue STI events. For mutex/semaphore bounce and issue detail, see **Mutex / Semaphore pairing** above.

Interval Analysis

Pairs **interval_start** / **interval_stop** STI events into measurable code regions. Each interval **id** gets an **Interval N** row on the timeline (horizontal task view) with colored span bars; the statistics table aggregates duration across all paired spans for that id.

Formula — for each paired instance j with start t_s and stop t_e :

$$\tau_j = t_e - t_s$$

Count is the number of paired spans in scope; Min / Avg / Max / p95 are over all interval durations τ_j .

What it tells you: Interval metrics measure **how long instrumented code regions take** — loop iterations, critical sections, or end-to-end handlers. Tight clusters in the distribution chart mean stable iteration time; outliers or a high **Max**

often mark contention, preemption inside the region, or pairing artefacts (see **Limitations** under Interval Analysis below). Compare interval ids to separate workloads (e.g. mutex stress vs lighter loops in the same trace).

Cross-task timing tip: `interval_start / interval_stop` pairing is scoped **per task id** (see [Pairing algorithm](#) below) — it measures elapsed time **within the same task**. To measure elapsed time **between two different tasks or ISRs** (e.g. a producer marks a tag on one task/core and a consumer picks it up on another), pairing by task id doesn't apply. Use a **tag channel** (`btf_traceTAG(id, value)`) instead: tag samples carry no task/id pairing, so consecutive samples on the same channel — regardless of which task emitted them — measure the cross-task interval directly. See [Tag Analysis → Interval tab](#).

BTF note field (last CSV column on each interval line):

Format	Example	Viewer pairing
Current firmware	1 tid:7 or 0 tid:0x8	Interval id + task id (decimal or 0x hex); same numeric id on different tasks does not cross-pair
Legacy	1	Full note string 1 only

Recorded by `traceINTERVAL_START(id) / traceINTERVAL_STOP(id)` in firmware — task id is captured automatically in `param2` and emitted in the note as `tid:....`. See [Binary → BTF dump mapping](#).

Column	Meaning
ID	Interval id from <code>traceINTERVAL_START(id) / traceINTERVAL_STOP(id)</code>
Label	Display name (Interval N)
Count	Number of paired start→stop spans in scope
Min / Avg / Max / p95	Interval duration statistics

Distribution chart — click any row:

- **Scatter:** x = interval stop time, y = interval duration.
- **Histogram:** distribution of interval durations.
- **Click a point** to jump to the **interval start** and add an annotation with duration/time notes.

Export HTML includes an **Interval instances** detail sub-table (longest first, up to 200 rows).

Interval 1 in `example-8cores.btf.gz` (1728 spans from eighteen `vCtxSwitch-Worker` tasks — `CS[11]-CS[28]` — sharing id 1 — each worker pairs via its own tid in the note):



Figure 20: Interval duration distribution for interval id 1 in `example-8cores.btf.gz`

Most spans cluster at short durations (tight yield loops); occasional high **y** points mark iterations that waited longer on the shared mutex or area semaphore. Compare ids **4-6** (shorter per-iteration work) vs **1-3** (heavier stress tests) in the same table.

Pairing algorithm ■ At parse time, the viewer collects all `interval_start` / `interval_stop` STI events, sorts them by time (start before stop at the same timestamp), and pairs them with a **per-key stack** (LIFO).

Note parsing (last BTF CSV field):

Note format	Pairing key	Timeline / stats row id
{id} tid:{task_id} (current firmware)	interval id + task_id	id only (Interval N)
{id} or other legacy text	full note string	same as note

When `tid:` is present, concurrent workers can share the same interval **id** without cross-pairing: task 7's `START(1)` only pairs with task 7's `STOP(1)`, even if task 8 uses id 1 at the same time. Legacy traces without `tid` keep the original behaviour (pair by note / id string only).

1. **interval_start** — push the event onto that key's stack.
2. **interval_stop** — pop the most recent unmatched start for that key and form one instance `[start_time, stop_time]`.
3. **interval_stop with an empty stack** — ignored (orphan stop).
4. **Unmatched starts after the trace ends** — counted internally but not shown in statistics or on the timeline.

An **Interval N** row appears only when at least one **start→stop** pair exists for that id. If firmware emits `interval_start` without matching `interval_stop` events (same id and `tid` when present), that id is omitted entirely.

Example — interval id with no paired spans:

Interval id	interval_start	interval_stop	Viewer
0	50	50	Interval 0
1	100	0	<i>(no row — nothing to pair)</i>
2	50	50	Interval 2

If firmware emits `interval_start` for an id but never records matching `interval_stop` events (same id and `tid` when present), that id is omitted from the time-

line and statistics. Ensure every `traceINTERVAL_START(id)` has a corresponding `traceINTERVAL_STOP(id)` with the same task id in the note when `tid:` is used.

This is **LIFO (last-in, first-out) nesting** within each pairing key.

```
# Nested on one task (same pairing key)
```

```
START(id=2)          stack: [A]
  START(id=2)         stack: [A, B]
  STOP(id=2)  →      pairs B→B  (inner span)
STOP(id=2)    →      pairs A→A  (outer span)
```

```
# Two tasks, same interval id, with tid in note
```

```
Task 7: START  note=1 tid:7
Task 8: START  note=1 tid:8
Task 7: STOP   note=1 tid:7  → pairs with task 7 start only
Task 8: STOP   note=1 tid:8  → pairs with task 8 start only
```

Result: **two** instances — one for the inner region, one for the outer region — each with the correct duration. **Min / Avg / Max / p95** in the statistics table are computed over **all** paired instances whose time range overlaps the active scope (full trace or cursor range). Timeline display uses a separate rule (see below).

Limitations ■ **1. Concurrent overlap with the same id (cross-pairing) — legacy traces without tid**

When the BTF note has **no** `tid:{task_id}` suffix, pairing uses the note string only. The stack algorithm then assumes stops arrive in the **reverse order** of starts. That holds for true call-stack nesting on one thread, but **not** when several tasks use the **same interval id** at the same time.

```
Task on Core_1: START(2) ————— STOP(2)
Task on Core_2:   START(2) ————— STOP(2)
Event order:      S1          S2          S1          S2
                  └─ stack pairs S2 with S1's STOP (wrong)
```

When starts and stops interleave across cores, the viewer can pair one task's **start** with another task's **stop**. That produces:

- **Count** — still the number of stop events that found a start on the stack (often equals the number of loop iterations).

- **Min / Avg / Max / p95** — can be **wrong**: bogus **very long** spans inflate max and avg; the distribution chart shows outlier points far above the real iteration time.

example-4cores.btf.gz is recorded **with** tid in each interval note, so ids **1-4** pair correctly across parallel workers. **Legacy** traces that omit tid may still cross-pair when several tasks share one id:

Interval id	Test (see Demo/examples/freertos_test/main.c)	Without tid — typical symptom
1	Context-switch / mutex stress (vCtxSwitchWorker)	Inflated Max / outlier scatter points
2	Mutex workers (vMutexWorker)	Same
3	Area-semaphore workers	Same
4	Nested interval stress	Same

For legacy traces, treat **short-duration clusters** in the scatter/histogram as the meaningful iteration times; treat isolated **very large** points and the table **Max** as pairing artefacts unless you have verified LIFO ordering for that id.

2. SMP task migration — why pairing is not done by core

The viewer pairs by **interval id** (and **task id** when tid is present), not by the **core** field on each STI event. A core-based matcher might seem attractive when several tasks share one id, but on **SMP** it is **not reliable**:

- A single FreeRTOS task can **migrate** between cores while an interval is open (START on Core_0, body runs, STOP on Core_2 after migration).
- Preemption and scheduling can also make the “running core” at START/STOP differ from the core where most of the work ran.
- Matching START and STOP by core would then **fail to pair** valid spans or would **split one logical interval** into orphan events.

Task A (may migrate):

START(2) on Core_0 ... runs on Core_0, then Core_1 ... STOP(2) on Core_1

Core-based match: no single core owns both ends → broken or missed pair

Id + tid stack: pairs correctly if no other task uses id 2 concurrently

So the viewer does **not** use core as a pairing key. With **tid in the note**, several tasks may share the same interval id safely; without tid, prefer **one interval id per logical scope** or accept LIFO-only pairing. Core columns on paired instances (start_core / stop_core in the parsed data) are informational only.

3. True time nesting vs concurrent overlap

Situation	Pairing correct?	Statistics meaningful?
Nested START/STOP on one thread (LIFO order)	Yes	Yes — each nesting level is a separate instance
Concurrent tasks, different ids	Yes	Yes — ids are independent
Concurrent tasks, same id, with tid in note	Yes	Yes — per-task pairing keys
Concurrent tasks, same id, no tid (legacy)	Often no	Count ok; min/avg/max may be skewed
Start without matching stop (crash / trace cut-off)	Partial	Unmatched starts excluded from stats

4. Timeline bars vs statistics count

The statistics table and distribution charts use **every** paired instance in scope. The timeline draws **top-level spans only**: if instance B's [start, stop] lies entirely inside instance A's range (same id), only A is drawn. This is a **display** rule only; it does not change **Count** or min/avg/max.

View	What you see
Statistics → Count	All paired spans (including nested and cross-paired)
Distribution chart	One point per paired span (same set as Count)
Timeline → Interval N row	Non-nested spans only (fully contained children hidden)

Typical timeline bar counts for the full `example-4cores.btf.gz` trace:

Interval id	Statistics Count	Timeline bars (top-level)
1	480	5
2	480	7
3	240	1
4	480	1

Interval 2 on the timeline: **7** bars (one long outer span plus six shorter top-level spans), while the table still reports **Count = 480**. Use the distribution chart (click a scatter point) or a narrow cursor range to jump to one specific paired instance.

5. Instrumentation guidance

To get reliable per-task interval statistics:

- **Preferred (firmware in this repo):** use `traceINTERVAL_START(id)` / `traceINTERVAL_STOP(id)` — the logger records **tid:{task_id}** in the BTF note so parallel workers can share the same numeric id.
- **Legacy traces** without `tid`: use a **distinct interval id per task** (or per logical scope), not one shared id across parallel workers.
- For nested regions on a single thread, reuse the same id — LIFO pairing matches `START/STOP` nesting within each pairing key.
- Do **not** rely on **core** to disambiguate pairs on SMP: tasks can **migrate** between `START` and `STOP` (see limitation 2 above).
- Orphan stops (stop without start) are dropped; orphan starts at end of trace are not counted in the table.

Timeline rendering ■ Interval bars are drawn as **solid** spans in the interval's colour. **Start** and **stop** events are marked with vertical tick lines (solid at start, dashed at stop) on the interval row.

Tag Analysis ■

Aggregates numeric samples from the 8 general-purpose STI **tag** channels (`tag0_event` ... `tag7_event`, plus the unindexed `tag_event`) — free-form values

emitted by firmware via `btf_traceTAG(id, value)` for any application-defined metric that doesn't fit an existing STI channel (queue depth, ADC reading, free heap, sensor reading, etc.). One row appears per channel that has at least one sample.

Formula — over all sample values v_k on a channel in scope: **Count** is the number of samples; **Min** / **Avg** / **Max** are the usual statistics; **p95** is the 95th-percentile value.

What it tells you: Unlike the other metric tables, the tag y-axis is the **raw application value**, not a duration — so what it means depends entirely on what firmware puts in the channel. A widening spread or a rising trend in the scatter plot can flag a slow memory leak (free heap), growing backlog (queue depth), or drifting sensor reading; a tight cluster around **Avg** with occasional **p95/Max** outliers is normal sampling noise.

Column	Meaning
Channel	Display name (Tag 0 ... Tag 7, or Tag for the unindexed tag_event)
Count	Number of samples in scope
Min / Avg / Max / p95	Sample value statistics (not a time duration)

Distribution chart — click any row:

- **Scatter:** x = sample time, y = tag value.
- **Histogram:** distribution of tag values.

Interval tab (time between samples)  The distribution popup has two in-dialog tabs — **Value** (above) and **Interval** — mirroring the **Core Migrations** Dwell/Rate/Gap tabs. Click **Interval** to switch the same scatter + histogram widgets to show the **elapsed time between consecutive samples** on that channel, regardless of which task or core emitted them.

Formula — for consecutive samples (sorted by time) k and $k-1$ on one channel:

$$\delta_k = t_k - t_{k-1}$$

Scatter: x = the later sample's time, y = δk (rendered as a duration, using the same adaptive time units as other metric charts). **Histogram:** distribution of δk with the usual min/avg/max/p95 reference lines and **CDF overlay**. Clicking a scatter point jumps to and annotates the later sample's time.

This is the **recommended way to measure elapsed time across two different tasks or ISRs**: emit `btf_traceTAG(id, marker)` from each side of the cross-task event (e.g. once when a producer task hands off, once when the consumer task/ISR observes it) and read the interval distribution's min/avg/max/p95 as the cross-task latency — no tid pairing is needed since tag samples are unpaired, timestamp-ordered markers.

tag0_event in `example-8cores.btf.gz` (2330 samples, min 8,144, avg $\approx$ 36,845, max 71,904, p95 41,936):

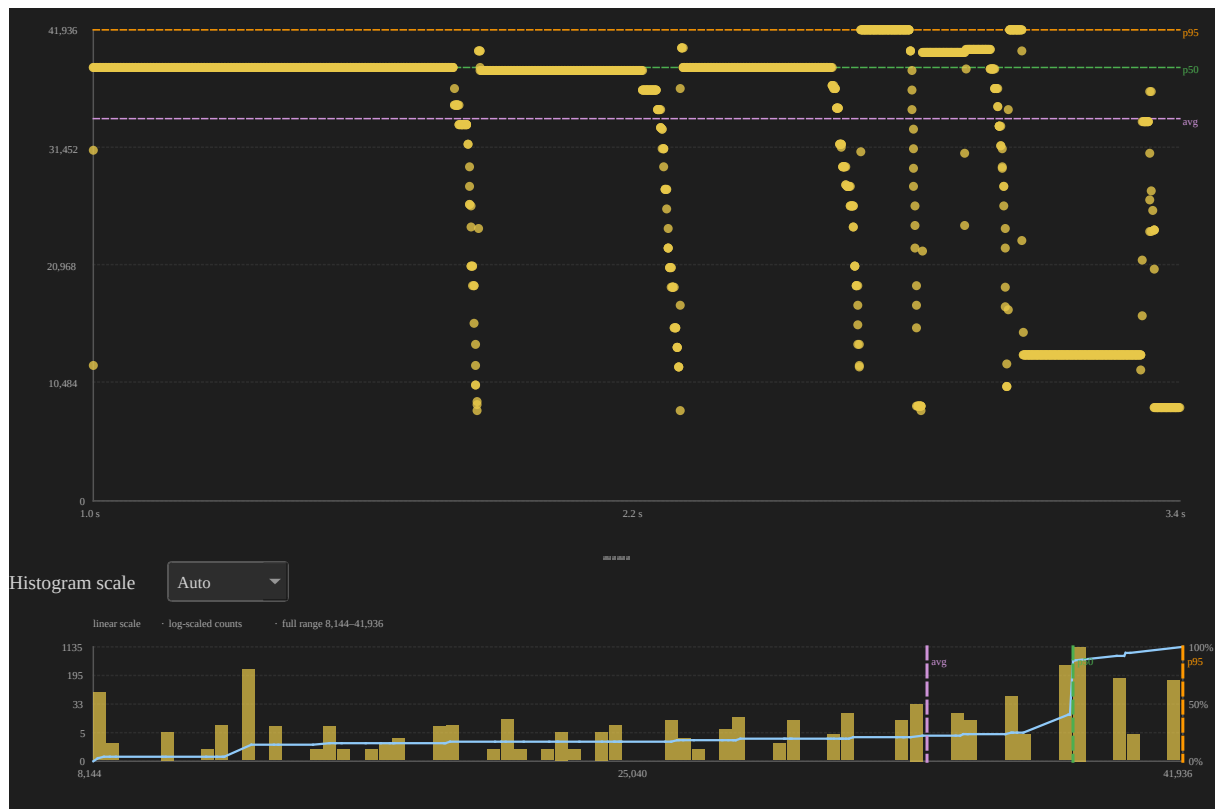


Figure 21: Tag value distribution chart for tag0_event in `example-8cores.btf.gz`

Shown only when the trace contains tag0_event ... tag7_event (or tag_event) STI samples.

Core migration and affinity

A **migration** is recorded when consecutive slices of the same task (merge-key) run on different cores. Migrations are detected at parse time from the segment timeline — there are no separate markers drawn on the timeline; use the **Core Migrations** table, the **Migration & Corridor Inspector** (toolbar **Heatmap**), **Trace Compare...**, or Find **Migrations** mode to inspect them.

Affinity, lifecycle, and deadline meanings are earlier in this chapter. This section is the deep dive for the migration tables, highlight / inspect steps, the corridor inspector, and **Trace Compare...**

Highlight a migrating task on the timeline

To see a hot migrating task in context (not only read rates in a table):

1. Stay in **Task View** (toolbar **Task**).
2. Click the task label (e.g. CS[22]) to **lock-highlight** it — other tasks stay on the timeline but gray out; do **not** use the legend filter.
3. Enable **Load** to see that task's CPU usage **per core** under the timeline (see **CPU Load**).
4. Optionally place cursors around the burst and enable **Limit to C1-Cn** so **Statistics → Core Migrations** recomputes for that window. The inspector grid follows the **visible timeline viewport** independently.

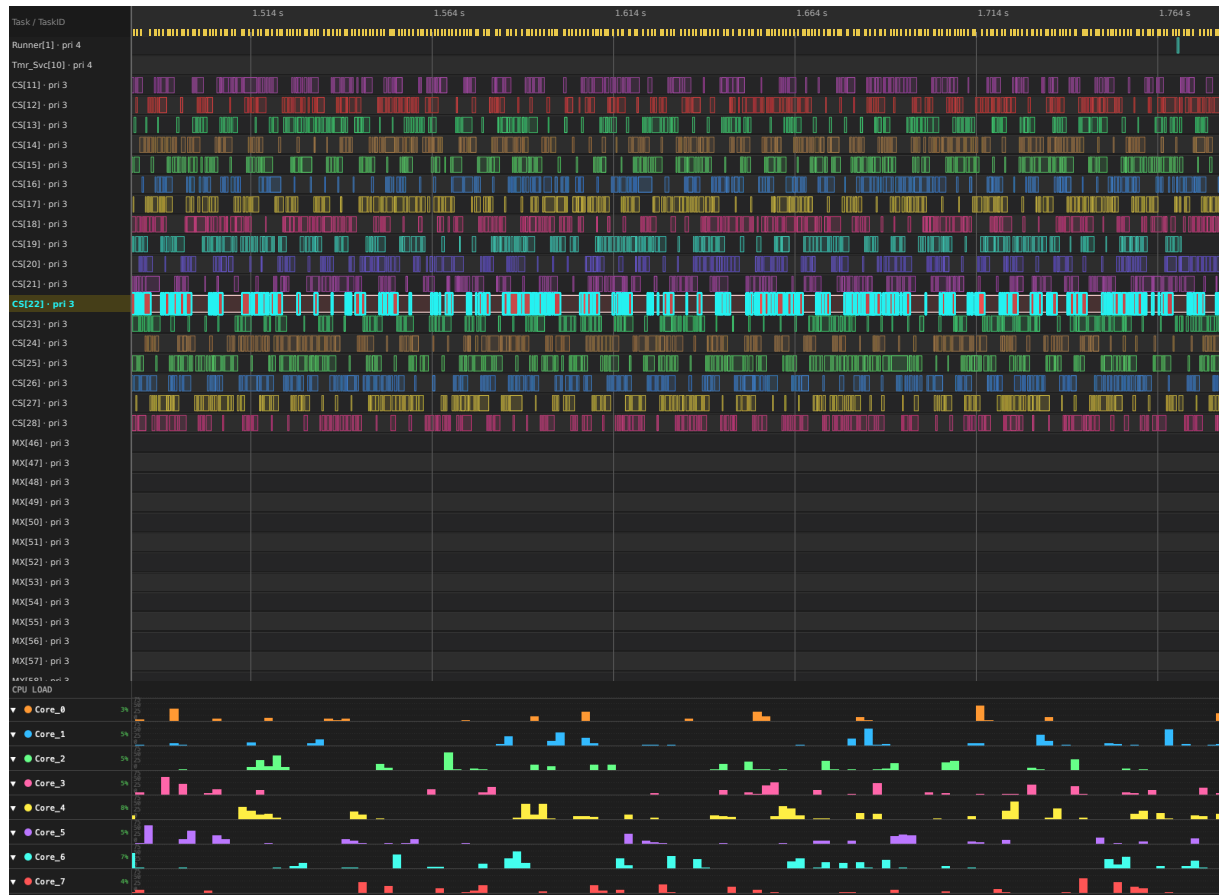


Figure 22: CS[22] highlighted in Task View with per-core CPU Load

CS[22] lock-highlighted in Task View; CPU Load shows that task's share on each core (example-8cores.btf.gz).

Headless example (full-span Task View + CPU Load for the same highlight):

```
python builds/btf_viewer.py snapshot ../tracedata/example-8cores.btf.gz \
-o /tmp/cs22-cpu-load.svg \
--view timeline --view-mode task --task "CS[22]" --cpu-load
```

Zoomed burst without the load strip:

```
python builds/btf_viewer.py snapshot ../tracedata/example-8cores.btf.gz \
-o /tmp/cs22-burst.svg \
--view timeline --view-mode task --task "CS[22]" \
--lo 1805000 --hi 1865000
```

Inspect migrations involving a specific core

After highlighting a hot task:

1. **Statistics → Core Migrations** — note **Primary** core, **Rate**, **Dwell**, **Ping** for that task. Click the row → **Dwell** / **Rate** / **Gap** charts.
2. **Core-Pair Migration Summary** — sort or scan pairs where that core is **From** or **To** (e.g. Core_5→Core_7) to see which neighbour absorbs the traffic and whether **Bounce %** is elevated. Click the row for Gap / Rate charts; use **Open Heatmap** / **Open Chord** from the dialog footer to open the unified inspector focused on that pair.
3. Toolbar **Heatmap** — corridor tree + time-bin grid: click a hot cell or expand a corridor to see contributing tasks, then **Inspect in Timeline** / double-click for Spotlight.
4. **Show topology** in the inspector (or pair-chart **Open Chord**) — same window with the topology sidebar expanded; hover outer (egress) / inner (ingress) rings or a ribbon to isolate flows.
5. **Core Time Breakdown** — click a core row to jump Core View focused on that core.
6. Headless CSV for a scoped window:

```
python builds/btf_viewer.py migrations ../tracedata/example-8cores.btf.gz \
  --lo 1805000 --hi 1865000 -o - | head
```

Legend panel: check **Migrated tasks only** to hide tasks that never left their first core.

Statistics → Core Migrations (collapsible section) — tasks that ran on two or more cores:

Migration rate — normalizes raw migration count against task active time and (when TICK STIs exist) scheduler ticks, so a task that migrates often relative to how much it runs stands out:

$$R_m = \frac{N_{\text{migrations},i}}{T_{\text{exec},i}}$$

The **Rate** column shows R_m as migrations per second of on-CPU time (e.g. 1.23/s). When the trace includes TICK events, it also shows migrations per **on-CPU** scheduler tick for that task (e.g. 2.785/tick) — TICK STIs that fall inside one of the task's slices in scope, not trace-wide tick count.

What it tells you: A high rate means the task is **bouncing** between cores (thrashing). For high-priority real-time tasks you ideally want this close to zero.

Average core dwell time — mean duration of each on-CPU stay before the task blocks, yields, or migrates:

$$\bar{T}_d = \frac{1}{N_{\text{slices}}} \sum_k d_k = \frac{T_{\text{exec},i}}{N_{\text{slices},i}}$$

Each slice dk is one switch-in episode (equivalent to averaging per-core dwell, $T_{\text{on}} / N_{\text{slices}}$, on each core the task visited).

What it tells you: If **Dwell** is extremely short (e.g. less than a few milliseconds or close to your system tick period), the scheduler is spending disproportionate effort moving the task between cores instead of letting it compute.

Column	Meaning
Task	Display name (Name[id])
Migr	Migration count in the current scope (full trace, or cursor range when Limit to C1-Cn is on)
Rate	Migration rate — /s of task active time; /tick = migrations per on-CPU TICK for this task in scope
Dwell	Average on-CPU slice duration (core dwell time) in scope
Cores	Distinct cores with on-CPU time or migrations in the current scope
Primary	Core with the most active time in scope, with its share (%)
Ping	Ping-pong migrations — three consecutive migrations A→B→A within 1 μs
STI±	Migrations with an STI event within ±500 ns
Gap after	Average off-CPU gap immediately after a migration
Gap other	Average blocking gap elsewhere for the same task

Click a row to open a **distribution chart** with three in-dialog tabs (switches the chart in place, no need to close and reopen):

- **Dwell** (default tab) — one point per on-core run: x = run start time, y = run

duration dk (same definition as **Average core dwell time** above, but per-sample instead of averaged).

- **Rate** — one point per migration after the first: x = migration time, y = time since the task's *previous* migration. Clusters of short gaps mean bursts of rapid bouncing; a flat, evenly spaced scatter means migrations are rare and spread out.
- **Gap** — one point per migration with a positive **Gap after** value: x = migration time, y = off-CPU gap immediately following that migration (the same samples averaged into the **Gap after** column; **Gap other** is not plotted here — open the **Blocking Time** chart for the same task instead, since it covers all off-CPU gaps).

Each tab's scatter + histogram uses the same **adaptive scaling** as other metrics charts (see [Metrics Distribution Charts](#)).

CS[22] in `example-8cores.btf.gz` (a context-switch stress task that migrates often):

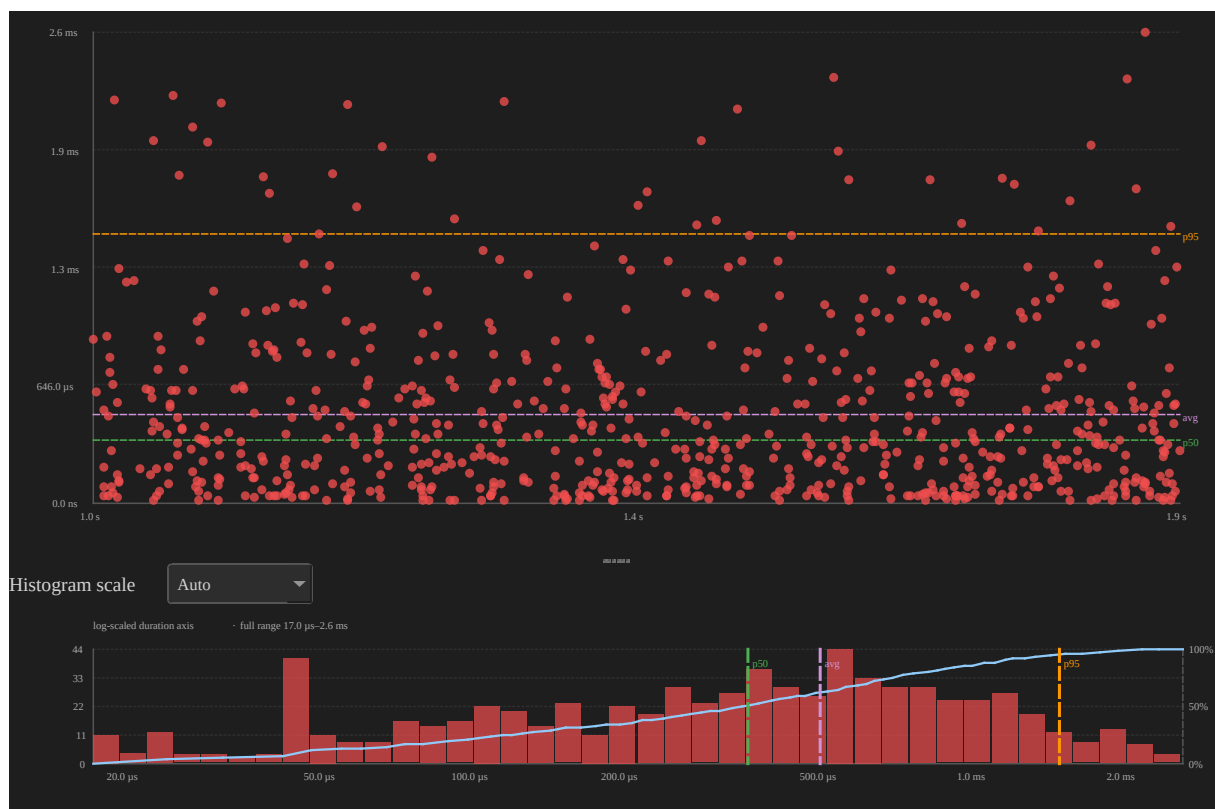


Figure 23: On-core dwell time distribution for CS[22] in `example-8cores.btf.gz`

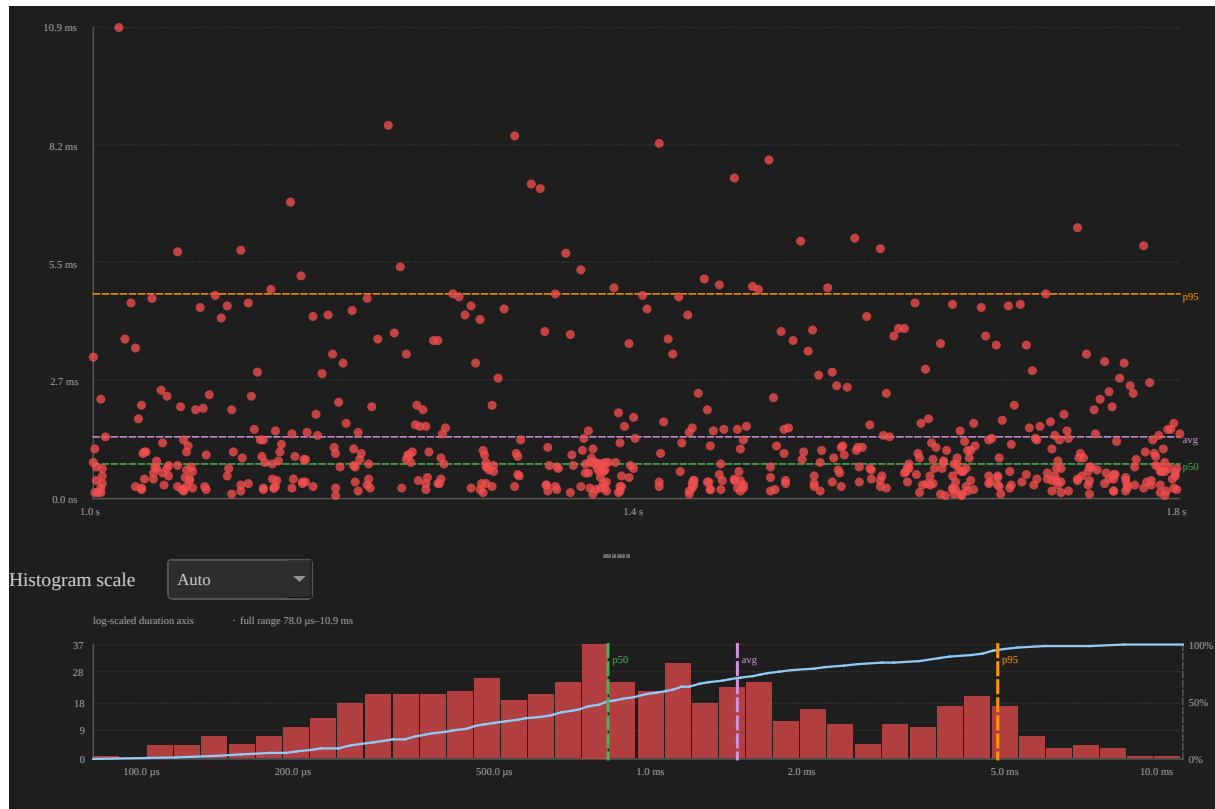


Figure 24: Time between migrations for CS[22] in example-8cores.btf.gz

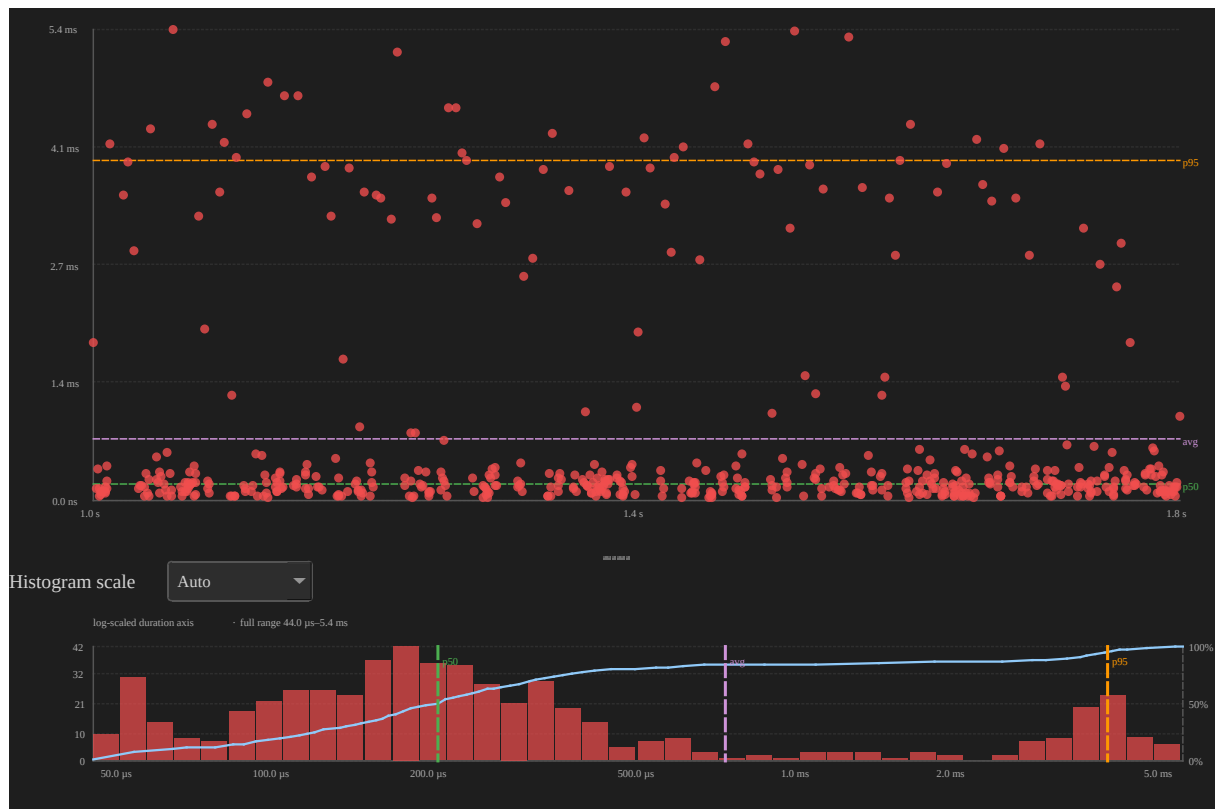


Figure 25: Post-migration gap distribution for CS[22] in example-8cores.btf.gz

Drag the resize handle below the table to show more or fewer rows.

Core-Pair Migration Summary ■

Directed corridors (From → To) aggregated across all tasks. Complements per-task **Core Migrations** (which task thrashing?) with pair-level volume and lock-bounce share.

Column	Meaning
From / To	Source and destination cores
Count	Directed migrations in scope
Bounces	Subset that occurred while a mutex was held across cores

Column	Meaning
Bounce %	$100 \times \text{Bounces} / \text{Count}$
Avg Gap	Mean post-migration off-CPU gap for that corridor

Click a row to open a **distribution chart** with two tabs:

- **Gap** (default) — one point per directed migration with a positive gap: x = migration time, y = post-migration gap (the samples behind **Avg Gap**). Lock-bounce events are drawn in **orange**; others use the source-core colour.
- **Rate** — one point per consecutive migration on the *same* directed pair: x = migration time, y = time since the previous hop on this corridor. Tight vertical bands mean bursty corridor traffic.

Dialog footer:

- **Open Heatmap** — open the Migration & Corridor Inspector focused on that pair (prefers **Lock Bounces Only** when Bounce % is elevated).
- **Open Chord** — same inspector with topology expanded and the pair highlighted.

Core_5 → **Core_7** in `example-8cores.btf.gz` (hottest corridor by Count):

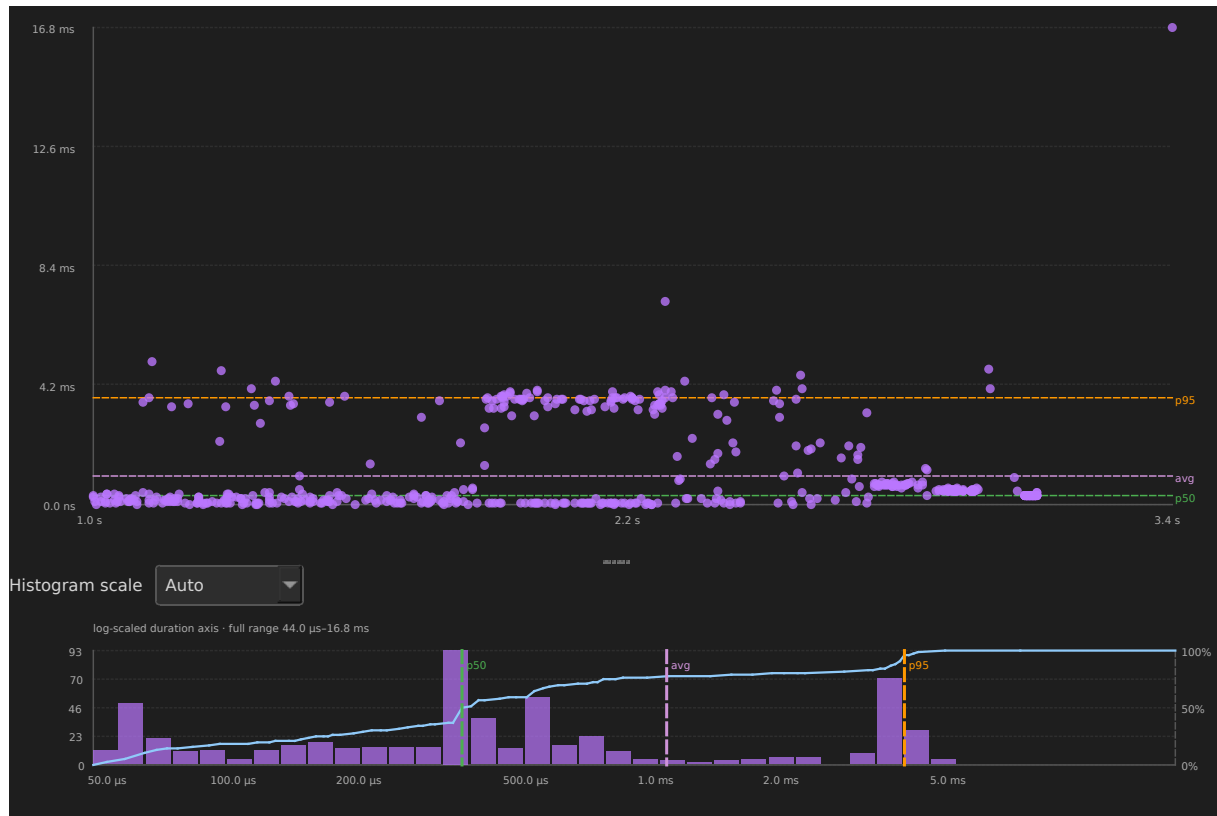


Figure 26: Post-migration gap for Core_5→Core_7

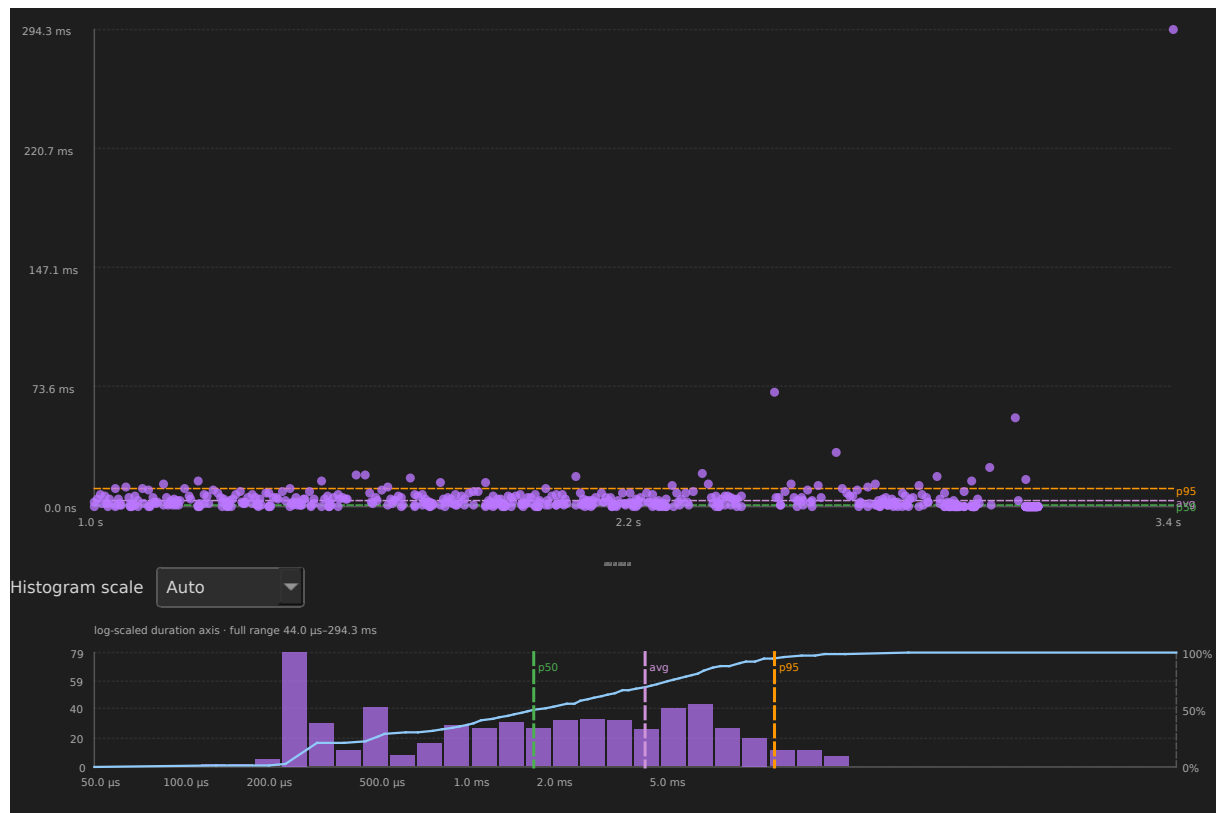


Figure 27: Time between pair migrations for Core_5→Core_7

Migration & Corridor Inspector ■

Toolbar **Heatmap** opens the inspector (Desktop + Web): corridor/task tree, time-bin grid, and mini-chord topology. **Open Chord** (core-pair charts) and **Show topology** inside the inspector expand the same window's topology sidebar.

Open

Toolbar **Heatmap** (2+ cores). Pair-chart **Open Chord** / inspector **Show topology** for the chord-first layout. Desktop: non-modal. Web: overlay keeps the timeline interactive. Tab switch closes the inspector.

Scope	Visible timeline viewport. Independent of Statistics Limit to C1-Cn . A Full view / Viewport view banner (same colors as the distribution chart) shows the current time range — Fit to window vs zoomed. Empty tree/grid shows <i>No migrations in scope</i> ; topology stays available.
Filter	Top corridors, Direction, Task filter. Lock Bounces Only when the trace has cross-core mutex holds.
Select	Click a tree row, grid cell, or chord ribbon. Double-click or Inspect in Timeline spotlights that bin (or task) with C1-C2. Toolbar All / Show all tasks clears the filter.
Query with AI...	Runs the Migration thrash template in the AI tab (same findings context as Statistics). If AI is disabled, opens Settings → AI .
> 16 cores	Tree groups by source; topology offers Circle ↔ Matrix. Dock topology Bottom or Right .

Example (example-8cores.btf.gz):

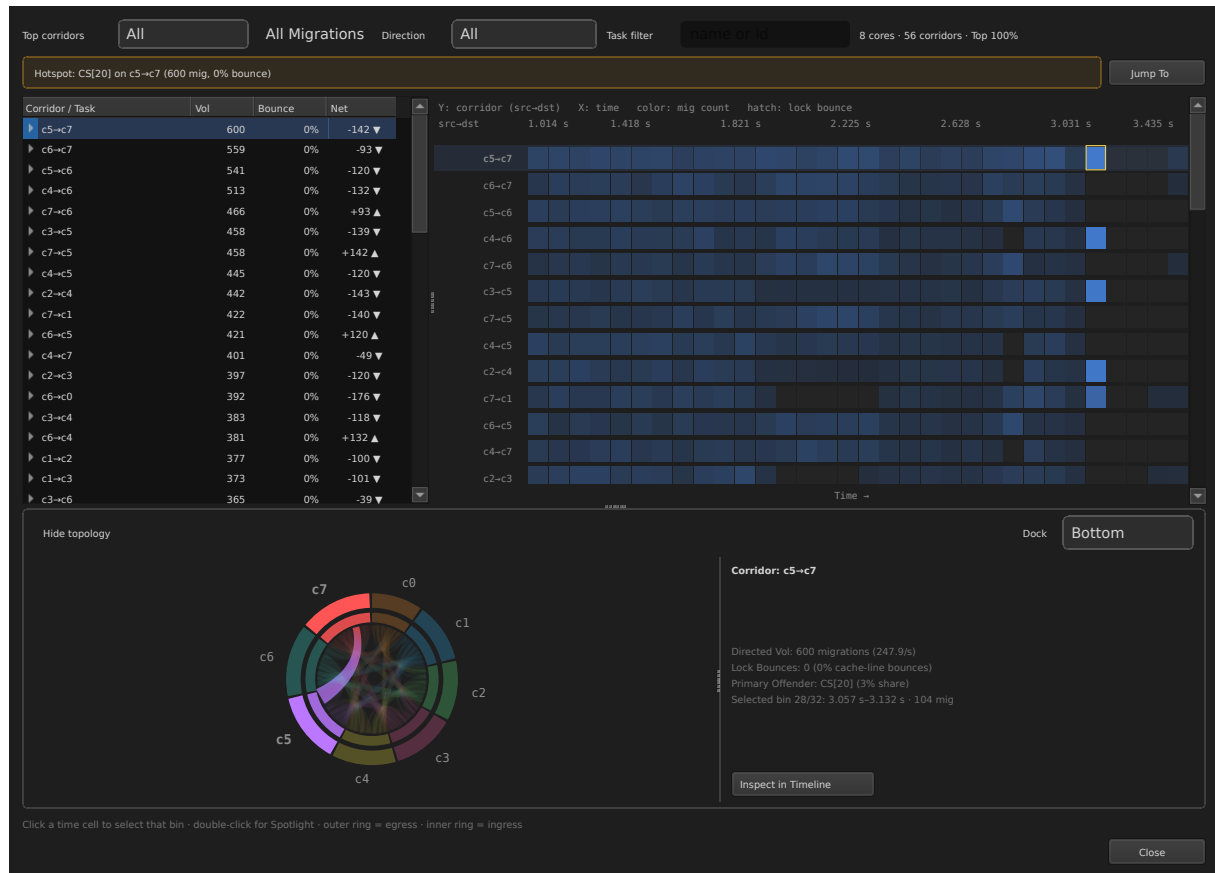


Figure 28: Migration & Corridor Inspector

Hover a ribbon for cN→cM: count in the footer. Per-task ping-pong / STI / gap-after aggregates are in Statistics **Core Migrations**.

`make -C BTFViewer update-images`

```
python builds/btf_viewer.py snapshot ../tracedata/example-8cores.btf.gz \
-o /tmp/migration.svg --view chord --width 1000 --height 720 --drill-row 0
```

Trace Compare... ■

Compare two traces you already have open side-by-side:

1. Open at least **two** .btf files (Desktop: **File** → **Open** adds a tab; Web: **Open** adds a tab in the bar under the toolbar).

2. Click toolbar **Compare** (right after **Analysis**; enabled when two or more tabs are loaded).
3. Choose **Trace A** and **Trace B** from the dropdowns.
4. Optionally check **Limit to each tab's cursor range** to compare metrics within C1-Cn on each trace (requires 2+ cursors per tab).
5. Switch between **Summary**, **Top Tasks**, **Core Util**, **Core Migrations**, **Execution**, **Blocking**, **Inter-Arrival**, **Preemption**, **Sync**, **Response**, and **Mutex** tabs.
6. Optionally click **Validate experiment...** to score expected vs actual deltas in the **AI** tab (host fills actual percents from this compare, including **Scope to cursors**), or **Query with AI...** to send the current Trace A / B tables (opens **Settings** → **AI** if the assistant is disabled).

By default, compare views use the **full trace**. With the cursor-range checkbox enabled, each side uses that tab's own cursor window independently.

Summary — high-level diff:

Metric	Notes
Span	Total trace duration (or cursor-range width when scoped)
Tasks / Segments / STI events	Counts
Context switches	Total across all cores
Core gap avg / max	Idle time between consecutive slices on each core
Migrations (total) / Migrated tasks	Core-migration counts
Load Balance Score / σ	Same Gini-based score and util stddev as Statistics Core Utilisation
Tick health / mode / count / missed	Same Trace Health (TICK) summary as Statistics

Each row shows Trace A, Trace B, and **Δ** (signed difference).

Top Tasks — top 10 user tasks by CPU% from each trace, unioned by display name (Name[id]). Tasks present in only one trace show — for the missing side.

Core Util — per-core utilisation % (IDLE/TICK excluded), A vs B with Δ.

Execution / Blocking / Inter-Arrival — top tasks by sample count with Runs/Gaps, Avg, Max, and Δ (aligned with the Statistics metric tables).

Core Migrations — same columns as the Statistics panel, compared side-by-side:

Column	Meaning
Task	Display name (Name[id])
Migr A / B	Migration count in scope for that trace
Δ	Difference (A – B)
Rate A / B	Migration rate label (/s and /tick) per trace
Rate Δ	Signed difference of migrations per second of on-CPU time (A – B)
Dwell A / B	Average on-CPU slice duration per trace
Dwell Δ	Signed difference of average dwell time (A – B)
Ping A / B	Ping-pong count in each trace
Cores A / B	Distinct cores used in scope
Primary A / B	Primary core and % of on-CPU time

Preemption / Sync — victim totals and sync-object aggregates (holds, issues, lock-bounce / affinity violations, mutex/sem/queue counts).

Response — per-task heuristic response P99 (ready→completion from adjacent slices) with A / B / Δ . **Mutex** — per-task mutex-wait totals. **Deadline misses** on Summary use the same **Settings → Display** task-deadline map as Statistics (0 when none are configured).

Use this to compare builds, configurations, or runs of the same workload without merging traces manually.

Use case: tickful vs tickless (performance and context switches) ■ Capture the **same workload** twice — once with a fixed tick (configUSE_TICKLESS_IDLE = 0) and once with FreeRTOS **tickless idle** (= 1) — then use **Trace Compare...** to quantify scheduler cost and application latency.

Sample pair: [tracedata/tickless-8cores.zip](#) (8-core demo; members tickful-8cores.btf + tickless-8cores.btf). Opening the zip in the GUI loads both as tabs; the headless CLI accepts the zip as a single compare input.

Capture (demo firmware)

```
# Fixed tick
make CORES=8 TICKLESS=0 run
cp tracedata/trace.btf tracedata/tickful-8cores.btf

# Tickless idle
make CORES=8 TICKLESS=1 run
cp tracedata/trace.btf tracedata/tickless-8cores.btf

# Optional: pack for GUI multi-tab open / CLI compare
zip -j tracedata/tickless-8cores.zip \
    tracedata/tickful-8cores.btf tracedata/tickless-8cores.btf
```

Keep STI **TICK** enabled on both builds and use the same suite / duration so Δ is meaningful.

Compare in the UI

1. Open the pair: `python builds/btf_viewer.py ../tracedata/tickless-8cores.zip` (two tabs), or open the two `.btf` files separately.
2. Optionally place matching cursor windows on the same busy (or idle) phase in each tab and enable **Limit to each tab's cursor range**.
3. Toolbar **Compare** → set Trace A / B labels (e.g. Tickful / Tickless).

What to read for performance and context switches

Compare tab / row	Why it matters
Summary → Context switches	Primary scheduler-activity cost between tick policies
Summary → Tick mode / Tick count / Tick health	Confirms config; tickful should favour lower CV when idle stretches dominate
Summary → Core gap avg/max, Load Balance Score / σ	Idle/busy structure and SMP balance
Summary → Migrations	Whether tick wake pattern changes cross-core bouncing

Compare tab / row	Why it matters
Execution (Max / p95)	Slice WCET and CPU-share shifts
Blocking (Max / p95)	Response-time impact under each policy
Preemption	Peer interference / tick-driven preemption differences
Top Tasks / Core Util	Who absorbs tick or wake-up overhead

CLI

```
# Zip with two .btf members (archive-root order → Trace A, Trace B)
python builds/btf_viewer.py compare ../tracedata/tickless-8cores.zip \
  -o /tmp/tick-policy.html --format html \
  --name-a Tickful --name-b Tickless
```

```
# Or two paths; optional shared busy/idle window (# timeScale units)
python builds/btf_viewer.py compare \
  ../tracedata/tickful-8cores.btf ../tracedata/tickless-8cores.btf \
  -o /tmp/tick-policy-busy.html \
  --name-a Tickful --name-b Tickless \
  --lo 1464000 --hi 1764000
```

Example Summary (tickless-8cores.zip, full-trace scope, A = Tickful, B = Tickless, $\Delta = A - B$):

Metric	Tickful	Tickless	Δ
Context switches	31,414	31,620	−206
Migrations (total)	19,018	18,440	+578
Tick count	2,561	2,611	−50
Load Balance Score	95 %	95 %	≈0
Span	2.421 s	2.444 s	−23 ms

On a **full** stress-suite capture (high core util), context-switch and tick counts can stay close — tick policy matters most in **idle-heavy** windows. Scope cursors (or --lo/-hi) around an idle phase (e.g. demo test 11) when measuring power-oriented

tickless gains; keep a busy CS window when checking that latency budgets still hold.

Known limitation — SMP tickless idle: on the bundled tickless-8cores.zip sample, test 11's TICK STI still fires every ~1 ms in *both* builds (no widened gaps in the tickless capture). Root cause is upstream in FreeRTOS-Kernel/tasks.c's prvGetExpectedIdleTime(), which forces the expected idle time to 0 whenever more than one task is ready at idle priority. Under SMP, a *running* task stays in its ready list (only xTaskRunState changes), so with configNUMBER_OF_CORES = 8 all 8 per-core IDLE tasks are simultaneously "ready", and tickless idle never actually engages. This is a vanilla-kernel heuristic gap, not a capture or viewer bug — expect tickless vs. tickful tick counts to stay close on SMP builds until the kernel's idle-ready-list check is made SMP-aware.

Interpretation

Observation	Typical reading
Tickless: TICKLESS mode / higher CV; tickful: TICK , CV $\ll$ 5 % (often clearer in idle-scoped windows)	Configurations captured correctly
Context switches ↓ on tickless in idle-heavy windows	Expected — suppressed idle ticks reduce scheduler wake-ups
Context switches similar on a fully busy CS phase	Tick policy has little effect when cores never idle
Blocking / Execution Max worse on one side	Prefer that policy only if the Δ fits latency budgets
Migrations ↑ with one policy	Re-check affinity; tick wake pattern can change placement

Prefer **tickless** when idle power matters and scoped busy-window metrics stay within budget. Prefer **tickful** when Trace Health must stay GOOD or soft real-time slices cannot tolerate tick stretching. Longer walkthrough: [WORKFLOWS.md §5.2](#).

Find → Migrations: lists migration boundary times; F3 / Shift+F3 jump between them (Desktop + Web).

Metrics distribution charts

In the **Statistics** panel, click any row in **Concurrent Core Active**, **Kernel Switch Overhead**, **Execution Time**, **Blocking Time**, **Dispatch / Scheduling Latency**, **Inter-Arrival**, **Core Migrations**, **Preemption Chain**, **Priority Inheritance**, **Interval Analysis**, or **Tag Analysis** to open a floating chart popup. In **Trace Health (TICK)**, use the **Tick Distribution...** button (bar-chart icon beside the mode badge when ≥ 2 ticks are in scope). **Core Migrations** popups additionally show in-dialog tabs (**Dwell / Rate / Gap**) and **Tag Analysis** popups show tabs (**Value / Interval**) to switch metrics without closing the chart.

- **Scatter plot** — each event plotted in trace time order so you can spot trends, bursts, or outliers.
- **Histogram** — adaptive bar chart of the value distribution:
 - **Auto scale** (default) picks **linear**, **p5-p95**, or **log duration** from the data spread so bars are not squeezed when min/max or outliers span a wide range.
 - **Histogram scale** dropdown (Desktop toolbar above histogram; Web above the chart): **Auto**, **Linear**, **p5-p95**, **Log duration**.
 - **Adaptive bin count** (Freedman-Diaconis, 12-80 bins) instead of a fixed 50-bin linear split.
 - **Overflow buckets** in p5-p95 mode — separate dimmed bars for values below p5 and above p95, with counts in the caption.
 - **Log-scaled counts** when one bin dominates (tall spike vs many small bars).
 - **Hover a bar** to see the bin range (or $< p5 / > p95$) and how many samples fall in that bucket.
 - **CDF overlay** — cumulative distribution curve on the histogram (see **CDF overlay** below).
 - Dashed reference lines for **avg**, **p5**, **p50**, and **p95**; caption shows the active scale and full min-max range.
- **Export PNG / SVG** — buttons in the chart footer save the current scatter + histogram.

The popup can be dragged, resized, and closed independently of the main window. If the chart is open, it **updates live** when you move cursors or toggle cursor-range scope. Each open trace tab remembers its own chart when you switch tabs.

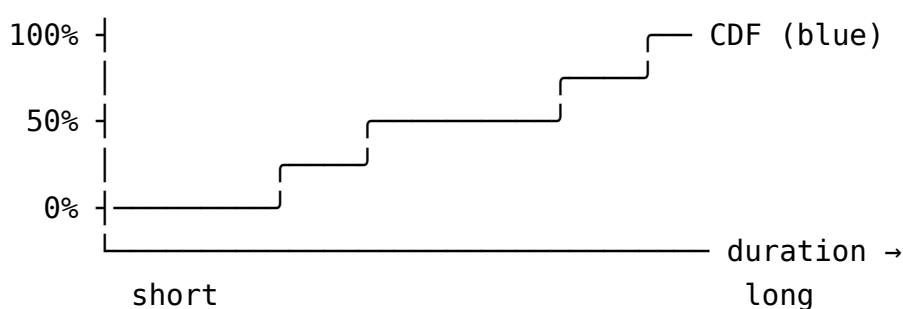
CDF overlay

Every metrics histogram includes a **cumulative distribution function (CDF)** drawn as a **blue line** over the bars. It answers a different question than the histogram alone:

View	Question it answers
Histogram bars	<i>How many samples fall in each duration bucket?</i>
CDF curve	<i>What fraction of samples are at or below a given duration?</i>

The CDF is an **empirical CDF (ECDF)**: for each sample in the active scope (full trace or cursor range), sorted by duration from shortest to longest, the curve plots (**duration → cumulative %**). There is one step per sample; when several samples share the same duration, the curve rises vertically at that x position.

How to read the chart



- **Horizontal axis (bottom)** — duration (same scale as the histogram bars: linear, p5-p95, or log, depending on **Histogram scale**).
- **Left vertical axis** — sample **count** per bin (bar height).
- **Right vertical axis** — cumulative **percent** (0%, 50%, 100% labels on a dashed guide line).

- **Curve direction** — starts at the **bottom-left** (0% of samples below the shortest value) and rises toward the **top-right** (100% of samples included). A **steep** rise means many samples cluster at similar short durations; a **gradual** rise means values are spread out.

Relating the CDF to table columns and reference lines

The dashed vertical markers on the histogram are single-number summaries; the CDF shows the **full** cumulative picture:

Marker / table column	On the CDF
p5 (cyan)	Curve crosses 5% on the right axis — 5% of samples are shorter than this duration.
p50 (green)	Curve crosses 50% on the right axis at the median duration.
p95 (orange)	Curve crosses 95% — 95% of samples are shorter than this duration.
avg (purple)	Shown as a vertical line; the CDF does not pass through a fixed “avg %” because the mean is not a percentile.

Example readings (Execution Time for a task with 100 slices):

- At duration D where the CDF is at **30%**, roughly **30 slices** (30%) finished in D or less — useful for “how often does this task complete within my deadline?”
- If the curve is already above **90%** while still on the **left half** of the chart, most runs are short and the tail is light.
- If the curve stays below **50%** until far right, the distribution is **wide or skewed** — the histogram bars alone may look crowded; switch **Histogram scale** to **Log duration** or **p5-p95** and use the CDF to see where the bulk of the mass sits.

Histogram scale and the CDF

The CDF always reflects the **same samples** as the bars; only the **x mapping** changes with the scale mode:

Histogram

scale	Effect on CDF
Auto / Linear	Duration mapped linearly from min to max.
p5-p95	Main curve spans the p5-p95 window; outliers appear in dimmed underflow / overflow buckets at the edges, and the CDF steps into those buckets at the corresponding percentiles.
Log duration	Short and long durations are spread across the axis; the CDF is easier to read when bars would otherwise pile up on the left.

The caption above the histogram (e.g. `log-scaled duration axis · full range 17 μs–975 μs`) always shows the **true** min-max range even when the axis is compressed or clipped.

When to use the CDF

- **Deadline / budget checks** — estimate what % of activations meet a time limit without reading individual scatter points.
- **Compare spread** — two tasks with similar **p50** can have very different CDF shapes (tight cluster vs long tail).
- **Skewed data** — after switching away from a crowded linear histogram, use the CDF with **p5** / **p50** / **p95** lines to see how much of the population sits below each marker.
- **Cursor-scoped analysis** — with **Limit to C1-Cn** enabled, the CDF recalculates for only the slices inside that window, same as the table and scatter plot.

The CDF is included in **Export PNG / SVG** from the plot dialog. It is not interactive (no click-to-jump); use the **scatter plot** above the histogram to jump to individual events.

Jump links: in Execution Time, Blocking Time, and Inter-Arrival tables, click **Min** / **Max** / **p95** / **p99** (dotted underline) to jump to that slice or gap and add an **annotation**. In **Dispatch / Scheduling Latency**, click **Min** or **Max**. Click any **distribution-chart** point to jump to that event and add an annotation without switching right-panel tabs (segment start for task metrics; tick timestamp for **Tick Distribution**; switch/concurrency timestamp for those plots; zoom + highlight for **Priority Inheritance** episodes; interval start for **Interval Analysis**). In Preemption Chain, the annotation is placed at the **preemptor segment** start. In **Mutex / Semaphore**, click any **Pairing issues** row to zoom to the running task segment

on that core, jump to the issue time, and add an annotation. **Timeline Anomalies** / **Worst Events** rows zoom and place C1-C2 on that episode.

Example plots from `tracedata/example-4cores.btf.gz` (4-core SMP trace, 67 tasks) are in [Statistics metric tables](#).

Documentation navigation

Document	Question answered
README.md	How do I use BTFViewer?
WORKFLOWS.md	How do I diagnose a problem?
STATISTICS.md	What does this measurement mean?
AI.md	How does AI-assisted investigation work?
